TRƯỜNG ĐẠI HỌC NGUYỄN TẤT THÀNH

KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN

HỌC MÁY VÀ ỨNG DỤNG

MÔ HÌNH DỰ ĐOÁN VÀ PHÁT HIỆN GIAN LẬN TRÊN THẺ TÍN DỤNG

Giảng viên hướng dẫn:	**Ts. Hà Minh Tân**
Sinh viên thực hiện:	**Lê Chí Kha**
MSSV:	**2200002544**
Khoá:	**2022 - 2026**
Ngành:	**CÔNG NGHỆ THÔNG TIN**
Chuyên ngành:	**KHOA HỌC DỮ LIỆU**

TPHCM, tháng 9 năm 2025

TRƯỜNG ĐẠI HỌC NGUYỄN TẤT THÀNH

KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN

HỌC MÁY VÀ ỨNG DỤNG

MÔ HÌNH DỰ ĐOÁN VÀ PHÁT HIỆN GIAN LẬN TRÊN THẺ TÍN DỤNG

Giảng viên hướng dẫn:	**Ts. Hà Minh Tân**
Sinh viên thực hiện:	**Lê Chí Kha**
MSSV:	**2200002544**
Khoá:	**2022 - 2026**
Ngành:	**CÔNG NGHỆ THÔNG TIN**
Chuyên ngành:	**KHOA HỌC DỮ LIỆU**

TPHCM, tháng 9 năm 2025

LỜI MỞ ĐẦU

Trong bối cảnh nền kinh tế số phát triển mạnh mẽ, các giao dịch tài chính điện tử ngày càng trở nên phổ biến, đặc biệt là giao dịch qua thẻ tín dụng. Tuy nhiên, song song với sự tiện lợi, tốc độ và khả năng kết nối toàn cầu, nguy cơ gian lận tài chính cũng ngày càng gia tăng với mức độ tinh vi và khó phát hiện hơn. Theo thống kê từ nhiều tổ chức tài chính quốc tế, thiệt hại do gian lận thẻ tín dụng gây ra mỗi năm lên tới hàng tỷ đô la Mỹ, ảnh hưởng nghiêm trọng đến lợi ích của cả ngân hàng và khách hàng. Trước thách thức đó, việc áp dụng các phương pháp học máy (Machine Learning) để xây dựng mô hình dự đoán và phát hiện gian lận trở thành một hướng tiếp cận hiệu quả. Các mô hình này có khả năng phân tích khối lượng dữ liệu lớn, tìm ra các mẫu hành vi bất thường, và đưa ra cảnh báo kịp thời, giúp giảm thiểu thiệt hại cũng như nâng cao độ an toàn của hệ thống thanh toán. Đồ án này tập trung nghiên cứu và triển khai " **MÔ HÌNH DỰ ĐOÁN VÀ PHÁT HIỆN GIAN LẬN TRÊN THẺ TÍN DỤNG "** dựa trên dữ liệu thực tế, thông qua việc xử lý dữ liệu, phân tích đặc trưng, và áp dụng nhiều thuật toán học máy . Từ đó, đánh giá và so sánh hiệu suất của các mô hình, đề xuất phương pháp tối ưu nhằm hỗ trợ các tổ chức tài chính nâng cao khả năng phòng chống gian lận. Với sự cố gắng hết sức tìm hiểu, phân tích và thiết kế nhưng chắc chắn không tránh khỏi được những thiếu sót. Em rất mong được sự thông cảm và góp ý của các quý thầy cô.

LỜI CẢM ƠN

Em chân thành cảm ơn Thầy hướng dẫn Ts. Hà Minh Tân , người đã hướng dẫn tận tình, đóng góp các ý kiến chuyên môn đồ án. Thầy đã động viên tinh thần để em cố gắng hoàn thành những nghiên cứu đặt ra. Thầy cũng cung cấp một số tài liệu liên quan đến đồ án mà em đang nghiên cứu và ân cần nhắc nhở em đến tiến độ thực hiện đồ án này.

Em cũng chân thành gửi lời cảm ơn đến các Thầy/Cô khoa Công nghệ thông tin trường Đại học Nguyễn Tất Thành đã giảng dạy, hướng dẫn, trang bị các kiến thức cho em trong những năm học vừa qua, từ các kiến thức cơ bản đến các vấn đề chuyên sâu.

Em xin gửi lời cảm ơn đến anh, chị, bạn bè, đồng nghiệp bằng nhiều hình thức khác nhau đã giúp đỡ em trong quá trình học tập tại trường cũng như trong thời gian hoàn thành đồ án.

Đặc biệt xin gửi lời cám ơn đến cha mẹ và anh chị em trong gia đình đã động viên tinh thần cũng như chia sẻ những khó khăn trong những năm qua. Em xin chân thành cảm ơn!

Sinh Viên Thực Hiện

(Ký Tên)

BM-ChT-

TRƯỜNG ĐẠI HỌC NGUYỄN TẤT THÀNH **TRUNG TÂM KHẢO THÍ**	**KỲ THI KẾT THÚC HỌC PHẦN HỌC KỲ .III. NĂM HỌC .2024 – 2025.**

PHIẾU CHẤM THI TIỂU LUẬN/ĐỒ ÁN

Môn thi: CÔNG NGHỆ KHOA HỌC DỮ LIỆU

Lớp học phần: 22DTH4B

Nhóm sinh viên thực hiện :

1. Lê Chí Kha — Tham gia đóng góp: 50%
2. Đỗ Kim Anh — Tham gia đóng góp: 50%

Ngày thi: 08/09/2025 Phòng thi: L.510

Đề tài tiểu luận/báo cáo của sinh viên : **MÔ HÌNH DỰ ĐOÁN VÀ PHÁT HIỆN GIAN LẬN TRÊN THẺ TÍN DỤNG**

Phần đánh giá của giảng viên (căn cứ trên thang rubrics của môn học):

Tiêu chí (theo CĐR HP)	**Đánh giá của GV**	**Điểm tối đa**	**Điểm đạt được**
Cấu trúc của báo cáo			
Nội dung			
• Các nội dung thành phần			
• Lập luận			
• Kết luận			
Trình bày			
TỔNG ĐIỂM			

Giảng viên chấm thi

(ký, ghi rõ họ tên)

MỤC LỤC

LỜI MỞ ĐẦU iii

LỜI CẢM ƠN iv

MỤC LỤC vi

DANH MỤC HÌNH x

KÍ HIỆU CÁC CỤM TỪ VIẾT TẮT xiii

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI 1

1.1. Bối cảnh và lý do chọn đề tài. 1

1.2. Mục tiêu nghiên cứu. 2

1.3. Phạm vi nghiên cứu. 3

1.4. Phương pháp nghiên cứu. 3

1.5. Cấu trúc báo cáo 4

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT 6

2.1. Khái niệm về gian lận thẻ tín dụng. 6

2.2. Nguyên tắc phát hiện gian lận. 8

2.3. Dữ liệu mất cân bằng trong phát hiện gian lận. 9

2.4. Các thuật toán học máy trong phát hiện gian lận. 10

2.4.1. Thuật toán Random Forest (RF): 10

2.4.2: Thuật toán AdaBoost (Adaptive Boosting): 13

2.4.3: Thuật toán CatBoost: 15

2.4.4: Thuật toán: 17

2.4.5: Thuật toán LightGBM (Light Gradient Boosting Machine): 19

2.5: Các chỉ số đánh giá mô hình. 21
2.5.1: Confusion Matrix (Ma trận nhầm lẫn): 21
2.5.2: Accuracy (Độ chính xác): 22
2.5.3: Precision (Độ chính xác dự đoán dương): 22
2.5.4: Recall (Độ bao phủ / Độ nhạy): 22
2.5.5: F1-score: 22
2.5.6: ROC Curve và AUC: 23
2.5.7: Lý do sử dụng nhiều chỉ số: 23
CHƯƠNG 3: DỮ LIỆU VÀ PHƯƠNG PHÁP THỰC HIỆN 24
3.1. Nguồn dữ liệu. 24
3.2. Mô tả tập dữ liệu. 24
3.3. Thiết lập môi trường và chuẩn bị dữ liệu. 25
3.3.1. Nền tảng thực hiện: 25
3.3.2. Môi trường thực nghiệm: 26
3.3.3. Triển khai mô hình: 26
3.3.4. Xem tổng quan dữ liệu: 27
3.3.5. Kiểm tra dữ liệu bị thiếu và mất cân bằng dữ liệu: 27
CHƯƠNG 4: KHÁM PHÁ VÀ PHÂN TÍCH DỮ LIỆU 29
4.1. Các biểu đồ giao dịch theo thời gian. 29
4.2. Giá trị của các giao dịch. 34
4.3. Sự tương quan giữa các đặc trưng. 36
4.3.1. Biểu đồ hồi quy giữa các biến và Amount: 37
4.3.2. Biểu đồ mật độ phân bố đặc trưng: 40

CHƯƠNG 5: XÂY DỰNG CÁC MÔ HÌNH DỰ ĐOÁN.....41
5.1. Chuẩn bị dữ liệu cho mô hình.....41
5.1.1. Xác định các biến dự báo và biến mục tiêu:.....41
5.1.2. Chia dữ liệu thành tập huấn luyện, tập kiểm tra và tập xác thực: 41
5.2. Mô Hình RandomForestClassifier. 42
5.2.1. Xác định tham số mô hình:.....42
5.2.2. Tầm quan trọng của các đặc trưng.....42
5.2.3. Ma trận nhầm lẫn: 43
5.3.1. Chuẩn bị mô hình và huyến luyện mô hình: 44
5.3.2: Tầm quan trọng của đặc trưng:.....44
5.3.3. Ma trận nhầm lẫn: 45
5.4. Mô Hình CatBoostClassifier.....46
5.4.1. Chuẩn bị mô hình và huyến luyện mô hình: 46
5.4.2. Tầm quan trọng của các đặc trưng:.....46
5.5. Mô Hình XGBoost. 48
5.5.1. Chuẩn bị mô hình và huyến luyện mô hình: 48
5.5.2. Tầm quan trọng của các đặc trưng:.....49
5.6: Mô Hình LightGBM.....49
5.6.1. Xác định tham số và chuẩn bị mô hình:.....49
5.6.2. Đặc trưng quan trọng của mô hình: 50
5.6.3. Huấn luyện và đánh giá mô hình bằng kỹ thuật kiểm định chéo: 50
CHƯƠNG 6: SO SÁNH KẾT QUẢ VÀ ĐÁNH GIÁ 53
6.1. Kết quả huyến luyện mô hình. 53

6.1.1. Mô hình Random Forest:.....53
6.1.2. Mô hình AdaBoost:.....53
6.1.3. Mô Hình CatBoostClassifier:.....54
6.1.4. Mô hình XGBoost:.....54
6.1.5. Mô hình LightGBM:.....54
6.2. Đánh giá mô hình tối ưu.....55
CHƯƠNG 7: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....56
7.1. Kết luận.....56
7.2. Hướng phát triển.....57
TÀI LIỆU THAM KHẢO.....58

DANH MỤC HÌNH

Hình 1.1: Fraud Detection là gì? 1

Hình 1.2. Sử dụng công cụ điện tử để ăn cắp thông tin thẻ 2

Hình 2.1: Tấn Công Phishing Nhắm Vào Người Dùng Mạng Xã Hội 6

Hình 2.2: Nhiều ngân hàng khẳng định không có dịch vụ rút tiền qua thẻ tín dụng. 6

Hình 2.3: Giao dịch giả mạo liên quan đến thẻ nội địa 7

Hình 2.4: Card Not Present (CNP): What It Is & How to Prevent CNP Fraud 7

Hình 2.5: Thuật toán Random Forest 10

Hình 2.6: Thuật toán AdaBoost (Adaptive Boosting) 13

Hình 2.7: Thuật toán CatBoost 15

Hình 2.8: Thuật toán XGBoost (Extreme Gradient Boosting) 17

Hình 2.9: Thuật toán LightGBM (Light Gradient Boosting Machine) 19

Hình 3.1: Đọc và kiểm tra nhanh dữ liệu 25

Hình 3.2: Giao diện chính của colab 26

Hình 3.3: Chỉ định đường dẫn dữ liệu. 26

Hình 3.4: Tính toán các thông kê cơ bản. 27

Hình 3.5: Kiểm tra dữ liệu bị khuyết (missing values). 27

Hình 3.6: Biểu đồ lớp gian lận tín dụng 28

Hình 4.1: Biểu đồ mật độ phân bố thời gian giao dịch 29

Hình 4.2: Phân biệt giữa giao dịch thường và gian lận 30

Hình 4.3: Biểu đồ thể hiện tổng số tiền giao dịch theo từng giờ 30

Hình 4.4: Biểu đồ thể hiện tổng số giao dịch 31

Hình 4.6: Biểu đồ thể hiện số lượng giao dịch tối đa 32

Hình 4.7: Biểu đồ thể hiện số lượng giao dịch trung bình 33

Hình: 4.8: Biểu đồ thể hiện số lượng giao dịch tối thiểu 33

Hình 4.10: Biểu đồ phân tán (Scatter Plot) 35

Hình 4.11: Ma trận tương quan (Pearson) giữa các thuộc tính trong 36

bộ dữ liệu giao dịch thẻ tín dụng 36

Hình 4.12: Biểu đồ hồi quy biến giữa V20 và Amount 37

Hình 4.13: Biểu đồ hồi quy giữa V7 và Amount 38

Hình 4.14: Biểu đồ hồi quy giữa V2 và Amount 38

Hình 4.15: Biểu đồ hồi quy giữa V5 và Amount 39

Hình 4.16: Biểu đồ mật độ phân phối (KDE) các biến theo Class 40

Hình 5.1: Khai báo biến dự báo và biến mục tiêu 41

Hình 5.2: Chia tập dữ liệu 41

Hình 5.3: Khởi tạo và huấn luyện mô hình 42

Hình 5.4: Biểu đồ thể hiện tầm quan trọng của đặc trưng 43

Hình 5.5: Ma trận nhầm lẫn của mô hình 43

Hình 5.6: Khởi tạo và dự đoán mô hình 44

Hình 5.7: Biểu đồ tầm quan trọng của các đặc trưng trong mô hình AdaBoost 45

Hình 5.8: Ma trận nhầm lẫn của mô hình AdaBoost 45

Hình 5.9: Khởi tạo và dự đoán mô hình 46

Hình 5.10: Biểu đồ tầm quan trọng của đặc trưng (CatBoost) 47

Hình 5.11: Ma trận nhầm lẫn của mô hình CatBoost 47

Hình 5.12: Khởi tạo và dự đoán mô hình 48

Hình 5.13: Dự đoán kết quả của mô hình 48

Hình 5.14: Biểu đồ đặc trưng quan trọng trong mô hình XGBoost 49

Hình 5.15: Khởi tạo và dự đoán mô hình 49

Hình 5.16: Biểu đồ tầm quan trọng của đặc trưng trong LightGBM 50

Hình 5.17: Chỉ số AUC đánh giá khả năng phân biệt giữa giao dịch gian lận và không gian lận 51

Hình 5.18: Kết quả đánh giá của chỉ số AUC 52

Hình 6.1: Kết quả (AUC - Area Under Curve) của Random Forest 53

Hình 6.2: Kết quả (AUC - Area Under Curve) của AdaBoost 53

Hình 6.3: Kết quả (AUC - Area Under Curve) của CatBoost 54

Hình 6.4: Kết quả (AUC - Area Under Curve) của XGBoost 54

Hình 6.5: Kết quả (AUC - Area Under Curve) của LightGBM 54

KÍ HIỆU CÁC CỤM TỪ VIẾT TẮT

Viết Tắt	Ý Nghĩa
EDA	Exploratory Data Analysis
PCA	Principal Component Analysis
SMOTE	Synthetic Minority Over-sampling Technique
ROC	Receiver Operating Characteristic
AUC	Area Under the Curve
TP	True Positive
TN	True Negative
FP	False Positive
FN	False Negative
CNN	Convolutional Neural Network
LSTM	Long Short-Term Memory
GPU	Graphics Processing Unit
CPU	Central Processing Unit

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1. Bối cảnh và lý do chọn đề tài.

Trong thời đại công nghệ số phát triển mạnh mẽ, thanh toán điện tử đã trở thành một phần không thể thiếu trong đời sống hàng ngày. Thẻ tín dụng, với ưu điểm nhanh chóng, tiện lợi và an toàn, đang được sử dụng rộng rãi trên toàn thế giới. Chỉ với vài thao tác, người dùng có thể mua sắm trực tuyến, thanh toán dịch vụ hay giao dịch ở bất kỳ đâu, bất kỳ thời điểm nào. Tuy nhiên, song song với sự phát triển của công nghệ, các hành vi gian lận thẻ tín dụng cũng ngày càng tinh vi và khó phát hiện hơn. Tội phạm mạng có thể lợi dụng lỗ hổng bảo mật hoặc các thủ thuật giả mạo để chiếm đoạt thông tin và thực hiện giao dịch trái phép. Theo nhiều báo cáo tài chính quốc tế, mỗi năm gian lận thẻ tín dụng gây ra thiệt hại hàng tỷ đô la Mỹ, tác động nghiêm trọng đến cả người tiêu dùng lẫn các tổ chức tài chính. Vấn đề càng trở nên thách thức hơn khi dữ liệu giao dịch thường mất cân bằng nghiêm trọng — tỷ lệ giao dịch gian lận rất nhỏ so với giao dịch hợp lệ. Điều này khiến các phương pháp phát hiện truyền thống khó đạt hiệu quả cao. Thêm vào đó, yêu cầu phát hiện gian lận gần như theo thời gian thực để ngăn chặn kịp thời càng đặt ra áp lực lớn cho hệ thống giám sát.



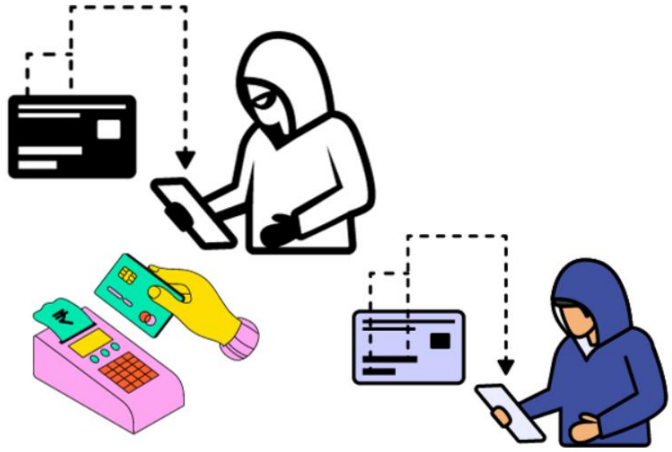
Hình 1.1: Fraud Detection là gì?

Xuất phát từ thực tế đó, nhóm lựa chọn đề tài "Mô hình dự đoán và phát hiện gian lận trên thẻ tín dụng" trong môn học Machine Learning. Đây không chỉ là cơ hội để vận dụng kiến thức về phân tích dữ liệu, xử lý mất cân bằng, và triển khai các thuật toán

học máy, mà còn mang ý nghĩa thực tiễn cao. Thông qua đề tài, nhóm mong muốn xây dựng một mô hình có khả năng:

- Phát hiện sớm và chính xác các giao dịch gian lận.
- Tối ưu hiệu suất với dữ liệu mất cân bằng.
- Gợi mở giải pháp áp dụng trong thực tế nhằm giảm thiểu rủi ro tài chính.

Với ý nghĩa đó, đề tài vừa đáp ứng mục tiêu học thuật, vừa hướng đến giải quyết một vấn đề cấp thiết trong lĩnh vực tài chính – ngân hàng hiện nay.



Hình 1.2. Sử dụng công cụ điện tử để ăn cắp thông tin thẻ

1.2. Mục tiêu nghiên cứu.

Trong thời đại giao dịch số phát triển mạnh mẽ, việc phát hiện và ngăn chặn gian lận trong thanh toán thẻ tín dụng là vấn đề cấp thiết đối với cả ngân hàng và khách hàng. Đề tài này được thực hiện với các mục tiêu chính sau:

- Hiểu và phân tích dữ liệu giao dịch thẻ tín dụng:
  - Làm rõ cấu trúc dữ liệu, ý nghĩa của các đặc trưng.
  - Nhận diện các đặc điểm khác biệt giữa giao dịch hợp lệ và giao dịch gian lận.
- Xử lý và chuẩn hóa dữ liệu:
  - Làm sạch dữ liệu, xử lý giá trị thiếu, biến đổi dữ liệu để phù hợp với mô hình học máy.
  - Giải quyết vấn đề mất cân bằng dữ liệu, vốn là đặc trưng phổ biến của các bài toán phát hiện gian lận.

- Xây dựng và huấn luyện các mô hình học máy:
  - Thử nghiệm nhiều thuật toán khác nhau
  - Tối ưu tham số để nâng cao hiệu quả dự đoán.
- Đánh giá và so sánh hiệu suất mô hình:
  - Sử dụng các chỉ số như Accuracy, Precision, Recall, F1-score và AUC.
  - Lựa chọn mô hình có khả năng phát hiện gian lận tốt nhất.
- Đề xuất giải pháp cải thiện và hướng phát triển:
  - Đưa ra các phương án nâng cao độ chính xác và khả năng ứng dụng của mô hình trong thực tế.
  - Đề xuất hướng mở rộng nghiên cứu, ví dụ triển khai mô hình trong hệ thống giám sát giao dịch thời gian thực.

1.3. Phạm vi nghiên cứu.

Đề tài tập trung vào việc phát hiện gian lận trong giao dịch thẻ tín dụng dựa trên dữ liệu có sẵn, cụ thể là tập dữ liệu lịch sử các giao dịch đã được gán nhãn là hợp lệ hoặc gian lận. Phạm vi nghiên cứu chỉ giới hạn ở việc phân tích và xử lý dữ liệu dạng bảng với các đặc trưng số học (numeric features), không đi sâu vào các nguồn dữ liệu phi cấu trúc như văn bản hay hình ảnh. Quá trình thực hiện sẽ bao gồm các bước thu thập, tiền xử lý, phân tích, xây dựng và đánh giá mô hình dự đoán bằng các thuật toán học máy. Môi trường nghiên cứu được triển khai trên máy tính cá nhân với ngôn ngữ lập trình Python và các thư viện phổ biến như pandas, scikit-learn, XGBoost và LightGBM. Kết quả mô hình chỉ phản ánh khả năng dự đoán trên tập dữ liệu nghiên cứu, chưa được triển khai và kiểm thử trong môi trường giao dịch thực tế, do đó các đề xuất ứng dụng thực tiễn sẽ mang tính định hướng và tham khảo.

1.4. Phương pháp nghiên cứu.

Đề tài sử dụng phương pháp nghiên cứu định lượng, kết hợp phân tích dữ liệu và xây dựng mô hình dự đoán dựa trên thuật toán học máy. Quá trình nghiên cứu được triển khai theo các bước chính sau: đầu tiên, tập dữ liệu giao dịch thẻ tín dụng được nạp vào môi trường Python và tiến hành khám phá dữ liệu (EDA) nhằm hiểu rõ phân bố

các đặc trưng, tỷ lệ giao dịch gian lận và mối tương quan giữa các biến. Tiếp theo, dữ liệu được tiền xử lý bao gồm xử lý giá trị thiếu, chuẩn hóa, và đặc biệt là giải quyết vấn đề mất cân bằng lớp thông qua các kỹ thuật như undersampling hoặc oversampling. Sau khi dữ liệu đã được làm sạch, nhiều mô hình học máy khác nhau sẽ được xây dựng và huấn luyện, bao gồm Random Forest, LightGBM, XGBoost, CatBoost và AdaBoost. Trong quá trình huấn luyện, các siêu tham số (hyperparameters) sẽ được tối ưu bằng phương pháp thử nghiệm và đánh giá chéo (cross-validation) để cải thiện hiệu suất. Cuối cùng, các mô hình sẽ được so sánh dựa trên các chỉ số đánh giá như Accuracy, Precision, Recall, F1-score và AUC, từ đó lựa chọn ra mô hình có hiệu quả cao nhất và đề xuất hướng áp dụng vào hệ thống giám sát giao dịch thực tế.

1.5. Cấu trúc báo cáo

Đồ án được bố cục thành 7 chương chính, sắp xếp theo trình tự từ khái quát đến chi tiết, nhằm giúp người đọc dễ dàng theo dõi và nắm bắt được toàn bộ quá trình nghiên cứu:

- **CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI**
  Trình bày bối cảnh, lý do chọn đề tài, mục tiêu nghiên cứu, phạm vi nghiên cứu, phương pháp nghiên cứu và cấu trúc báo cáo.
- **CHƯƠNG 2. CƠ SỞ LÝ THUYẾT**
  Trình bày các khái niệm về gian lận thẻ tín dụng, nguyên tắc phát hiện gian lận, dữ liệu mất cân bằng, các thuật toán học máy và các chỉ số đánh giá mô hình.
- **CHƯƠNG 3. DỮ LIỆU VÀ PHƯƠNG PHÁP THỰC HIỆN**
  Giới thiệu nguồn dữ liệu, mô tả tập dữ liệu, nền tảng thực hiện và các bước chuẩn bị dữ liệu.
- **CHƯƠNG 4: KHÁM PHÁ VÀ PHÂN TÍCH DỮ LIỆU**
  Phân tích dữ liệu qua các biểu đồ thời gian, giá trị giao dịch, sự tương quan giữa các đặc trưng.

- **CHƯƠNG 5: XÂY DỰNG CÁC MÔ HÌNH DỰ ĐOÁN**
  Chuẩn bị dữ liệu cho mô hình, huấn luyện và đánh giá các mô hình Random Forest, AdaBoost, CatBoost, XGBoost và LightGBM.
- **CHƯƠNG 6: SO SÁNH VÀ ĐÁNH GIÁ KẾT QUẢ**
  Trình bày kết quả huấn luyện các mô hình, so sánh hiệu suất và lựa chọn mô hình tối ưu.
- **CHƯƠNG 7: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN**
  Tóm tắt kết quả đạt được, rút ra bài học kinh nghiệm và đề xuất hướng nghiên cứu, cải tiến trong tương lai.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Khái niệm về gian lận thẻ tín dụng.

Gian lận thẻ tín dụng (Credit Card Fraud) là hành vi sử dụng trái phép thông tin thẻ tín dụng của người khác nhằm thực hiện các giao dịch tài chính mà không có sự cho phép của chủ thẻ. Đây là một dạng tội phạm tài chính phổ biến trong kỷ nguyên giao dịch điện tử, gây thiệt hại nghiêm trọng cho cả khách hàng và các tổ chức tài chính. Các hành vi gian lận thẻ tín dụng có thể được thực hiện bằng nhiều hình thức khác nhau, tiêu biểu gồm:

- Đánh cắp thông tin thẻ thông qua các kỹ thuật như phishing (lừa đảo trực tuyến), skimming (sao chép dữ liệu từ dải từ hoặc chip trên thẻ), hoặc tấn công mạng để xâm nhập cơ sở dữ liệu lưu trữ thông tin thẻ.



Hình 2.1: Tấn Công Phishing Nhắm Vào Người Dùng Mạng Xã Hội

- Sử dụng trái phép thẻ bị mất hoặc bị đánh cắp để thanh toán hàng hóa, dịch vụ hoặc rút tiền mặt.



Hình 2.2: Nhiều ngân hàng khẳng định không có dịch vụ rút tiền qua thẻ tín dụng.

- Làm giả thẻ bằng cách sao chép dữ liệu từ thẻ thật và in lên thẻ trắng để thực hiện giao dịch bất hợp pháp.



Hình 2.3: Giao dịch giả mạo liên quan đến thẻ nội địa

- Tấn công giao dịch trực tuyến (Card-Not-Present fraud) khi kẻ gian chỉ cần thông tin số thẻ, ngày hết hạn và mã bảo mật để mua hàng qua mạng mà không cần thẻ vật lý.



Hình 2.4: Card Not Present (CNP): What It Is & How to Prevent CNP Fraud

Đặc điểm nổi bật của gian lận thẻ tín dụng là tỷ lệ xảy ra rất thấp so với tổng số giao dịch (thường <0.2%), nhưng hậu quả tài chính lại rất lớn. Điều này khiến việc phát hiện gian lận trở nên khó khăn, đặc biệt khi kẻ gian thường sử dụng các phương thức tinh vi để ngụy trang hành vi của mình sao cho giống với các giao dịch hợp lệ.

Trong bối cảnh thương mại điện tử và thanh toán trực tuyến phát triển mạnh, nhu cầu về các giải pháp tự động, chính xác và gần như theo thời gian thực để phát hiện gian lận ngày càng cấp thiết, và học máy (Machine Learning) đang là một hướng tiếp cận nổi bật để giải quyết vấn đề này.

2.2. Nguyên tắc phát hiện gian lận.

Phát hiện gian lận thẻ tín dụng là quá trình nhận diện các giao dịch bất thường so với hành vi giao dịch thông thường của chủ thẻ, nhằm đưa ra cảnh báo và ngăn chặn kịp thời các hành vi gian lận. Mục tiêu của quá trình này là đảm bảo an toàn cho hệ thống thanh toán, giảm thiểu thiệt hại tài chính và bảo vệ uy tín của tổ chức tài chính.

Nguyên tắc chung của phát hiện gian lận bao gồm các bước:

- Thu thập và phân tích dữ liệu giao dịch: Lưu trữ và phân tích lịch sử giao dịch để xác định các mẫu hành vi bình thường.
- So sánh hành vi hiện tại với hành vi lịch sử: Đánh giá xem giao dịch mới có nằm trong phạm vi hành vi bình thường hay không.
- Xác định và gắn nhãn bất thường: Nếu một giao dịch khác biệt đáng kể so với hành vi thông thường, hệ thống sẽ gắn cờ (flag) để cảnh báo hoặc yêu cầu xác minh.

- Các phương pháp phát hiện gian lận được chia thành hai nhóm chính:

Phương pháp truyền thống:

- Dựa trên luật (Rule-based): Sử dụng các quy tắc được định nghĩa thủ công, ví dụ "Giao dịch có giá trị trên 50 triệu VNĐ ở quốc gia khác trong vòng 24 giờ sẽ bị cảnh báo".
- Dựa trên thống kê: Phân tích phân bố dữ liệu và đặt ngưỡng phát hiện bất thường.
- Ưu điểm của nhóm này là dễ triển khai, dễ giải thích, nhưng hạn chế ở khả năng thích ứng với các phương thức gian lận mới.

Phương pháp dựa trên học máy (Machine Learning):

- Mô hình học từ dữ liệu lịch sử đã gán nhãn (hợp lệ/gian lận) để tự động nhận diện các mẫu bất thường.
- Có khả năng phát hiện các hành vi gian lận tinh vi mà phương pháp truyền thống khó nhận diện.

- Bao gồm cả học có giám sát (supervised learning) và học không giám sát (unsupervised learning), trong đó học có giám sát thường được áp dụng nhiều hơn khi dữ liệu đã có nhãn.

Điểm thách thức của phát hiện gian lận bằng học máy là vấn đề mất cân bằng dữ liệu (tỷ lệ gian lận rất nhỏ) và yêu cầu dự đoán gần như theo thời gian thực, đòi hỏi mô hình vừa chính xác vừa tối ưu về tốc độ xử lý.

2.3. Dữ liệu mất cân bằng trong phát hiện gian lận.

Trong bài toán phát hiện gian lận thẻ tín dụng, dữ liệu giao dịch thường có tỷ lệ mất cân bằng nghiêm trọng (class imbalance). Cụ thể, số lượng giao dịch hợp lệ (non-fraud, Class = 0) chiếm tỷ lệ áp đảo so với giao dịch gian lận (fraud, Class = 1). Ví dụ, trong bộ dữ liệu creditcard.csv được sử dụng trong nghiên cứu này, chỉ khoảng 0.17% giao dịch là gian lận, tức là trung bình 1 giao dịch gian lận trên hơn 500 giao dịch hợp lệ.

Vấn đề mất cân bằng dữ liệu gây ra một số khó khăn trong huấn luyện mô hình:

- Độ chính xác (Accuracy) gây hiểu nhầm: Nếu mô hình dự đoán tất cả giao dịch là hợp lệ, vẫn có thể đạt Accuracy trên 99%, nhưng hoàn toàn không phát hiện được giao dịch gian lận.
- Mô hình thiên lệch về lớp đa số: Các thuật toán học máy thường tối ưu cho độ chính xác tổng thể, dẫn đến việc bỏ qua lớp thiểu số.
- Khó đánh giá hiệu quả thật sự của mô hình: Trong dữ liệu mất cân bằng, các chỉ số như Accuracy không phản ánh đúng khả năng phát hiện gian lận, cần sử dụng các chỉ số khác như Precision, Recall, F1-score và AUC

Các kỹ thuật xử lý dữ liệu mất cân bằng:

- Undersampling: Giảm số lượng mẫu ở lớp đa số để cân bằng với lớp thiểu số.
  - Ưu điểm: Giảm kích thước dữ liệu, tăng tốc huấn luyện.
  - Nhược điểm: Có thể làm mất thông tin quan trọng.
- Oversampling: Tăng số lượng mẫu lớp thiểu số bằng cách sao chép hoặc tạo mẫu mới (ví dụ: SMOTE – Synthetic Minority Over-sampling Technique).

- Ưu điểm: Bảo toàn thông tin lớp đa số, cải thiện khả năng nhận diện lớp thiểu số.
- Nhược điểm: Có thể gây overfitting nếu tạo quá nhiều mẫu nhân tạo.

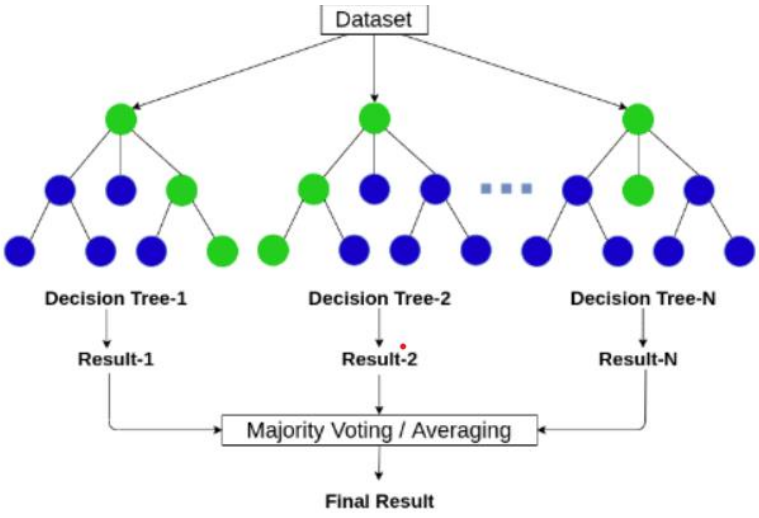
- Sử dụng thuật toán chuyên biệt: Một số mô hình như Balanced Random Forest hoặc XGBoost cho phép điều chỉnh trọng số (class weight hoặc scale_pos_weight) để tăng độ nhạy với lớp thiểu số.
- Kết hợp nhiều kỹ thuật: Áp dụng đồng thời undersampling và oversampling để vừa cân bằng dữ liệu vừa giữ thông tin quan trọng.

Giải quyết vấn đề mất cân bằng là bước quan trọng trong phát hiện gian lận, giúp mô hình đạt độ nhạy cao (Recall) mà vẫn duy trì độ chính xác (Precision) ở mức chấp nhận được, từ đó giảm thiểu rủi ro bỏ sót giao dịch gian lận.

2.4. Các thuật toán học máy trong phát hiện gian lận.

Phát hiện gian lận thẻ tín dụng đòi hỏi mô hình vừa chính xác, vừa xử lý nhanh trên lượng dữ liệu lớn, lại phải nhận diện được các mẫu hiếm (dữ liệu mất cân bằng). Trong phạm vi nghiên cứu này, các thuật toán được lựa chọn đều thuộc nhóm ensemble learning – kết hợp nhiều mô hình con để tăng độ chính xác và khả năng khái quát.

2.4.1. Thuật toán Random Forest (RF):



Hình 2.5: Thuật toán Random Forest

a. Nguyên lý.

Random Forest là một thuật toán thuộc nhóm bagging (Bootstrap Aggregating), kết hợp nhiều cây quyết định (Decision Trees) để tạo thành một mô hình mạnh mẽ hơn.

- Mỗi cây được huấn luyện trên một tập dữ liệu con được lấy mẫu ngẫu nhiên có hoàn lại (bootstrap sampling) từ tập dữ liệu gốc.
- Khi xây dựng mỗi nút của cây, chỉ một tập con đặc trưng được chọn ngẫu nhiên để tìm điều kiện phân chia tốt nhất.
- Kết quả dự đoán cuối cùng là bỏ phiếu đa số (cho phân loại) hoặc trung bình cộng (cho hồi quy).

b. Công thức dự đoán.

- Phân loại (Classification):

$$\hat{y} = \text{mode}\{h_t(x) \mid t = 1, 2, \ldots, T\}$$

- Hồi quy (Regression):

$$\hat{y} = \frac{1}{T} \sum_{t=1}^{T} h_t(x)$$

Trong đó:

- $h_t(x)$: dự đoán của cây thứ t
- T: tổng số cây trong rừng

c. Công thức tính độ quan trọng của đặc trưng (Feature Importance).

Một trong những cách phổ biến là Mean Decrease in Impurity (MDI):

$$\text{Imp}(f) = \frac{1}{T} \sum_{t=1}^{T} \sum_{n \in N_t(f)} p(n) \cdot \Delta i(n)$$

Trong đó:

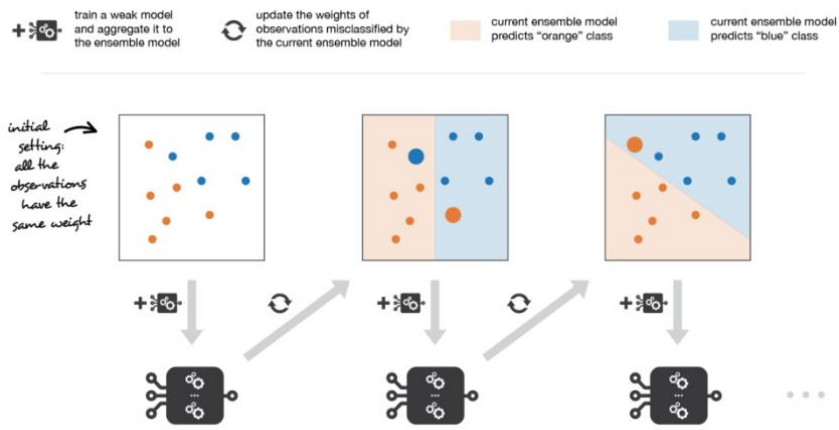
- $N_t(f)$: tập các nút trong cây t sử dụng đặc trưng f để phân chia
- $p(n)$: tỷ lệ số mẫu tại nút n
- $\Delta i(n)$: mức giảm độ hỗn tạp (*impurity*, ví dụ Gini hoặc Entropy) khi phân chia tại nút n

- Ưu điểm:
  - Hoạt động tốt với dữ liệu nhiều đặc trưng, kể cả dữ liệu nhiễu.
  - Có khả năng xác định mức độ quan trọng của đặc trưng (feature importance).
  - Ít bị overfitting hơn so với một cây quyết định đơn lẻ.
  - Không yêu cầu chuẩn hóa hoặc chuẩn bị dữ liệu quá phức tạp.
- Nhược điểm:
  - Mô hình thường có kích thước lớn ⇒ tốn bộ nhớ và thời gian dự đoán lâu.
  - Khó giải thích chi tiết quá trình dự đoán.
  - Với dữ liệu mất cân bằng mạnh, mô hình dễ thiên về lớp đa số nếu không xử lý hoặc điều chỉnh trọng số.

d. Ứng dụng trong phát hiện gian lận thẻ tín dụng.

- Được dùng như mô hình baseline do khả năng huấn luyện nhanh và hiệu quả tốt mà không cần tinh chỉnh nhiều.
- Kết hợp với class_weight = 'balanced', undersampling hoặc oversampling (SMOTE) để cải thiện khả năng phát hiện gian lận.
- Thường dùng để so sánh với các thuật toán boosting (LightGBM, XGBoost) nhằm lựa chọn mô hình tối ưu cho triển khai thực tế.

2.4.2: Thuật toán AdaBoost (Adaptive Boosting):



Hình 2.6: Thuật toán AdaBoost (Adaptive Boosting)

a. Nguyên lý.

AdaBoost là thuật toán thuộc nhóm boosting, hoạt động bằng cách kết hợp nhiều mô hình yếu (*weak learners*), thường là decision stump (cây quyết định có độ sâu 1), để tạo thành một mô hình mạnh hơn.

Quy trình hoạt động:

- Khởi tạo trọng số mẫu: Ban đầu, mọi mẫu dữ liệu đều có trọng số như nhau.
- Huấn luyện mô hình yếu: Ở mỗi vòng lặp, huấn luyện một mô hình yếu dựa trên trọng số hiện tại của các mẫu.
- Tính lỗi mô hình: Mẫu bị dự đoán sai sẽ được tăng trọng số, mẫu dự đoán đúng sẽ giảm trọng số.
- Kết hợp mô hình: Các mô hình yếu được kết hợp bằng trọng số alpha dựa trên độ chính xác của từng mô hình.
- Lặp lại quá trình đến khi đạt số vòng lặp M hoặc lỗi nhỏ hơn ngưỡng.

b. Công thức.

- Trọng số của mô hình yếu thứ m:

$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_m}{\varepsilon_m} \right)$$

Trong đó:

- ε_m: Tỷ lệ lỗi của mô hình yếu thứ m
- α_m: Trọng số mô hình trong tổ hợp

- Cập nhật trọng số mẫu:

$$w_i^{(m+1)} = w_i^{(m)} \cdot e^{\alpha_m \cdot I(y_i \neq h_m(x_i))}$$

Trong đó:

- $w_i^{(m)}$: Trọng số mẫu i ở vòng m
- $h_m(x_i)$: Dự đoán của mô hình yếu thứ m
- $I(y_i \neq h_m(x_i))$: Hàm chỉ báo, bằng 1 nếu dự đoán sai, 0 nếu đúng

- Dự đoán cuối cùng:

$$H(x) = \text{sign} \left(\sum_{m=1}^{M} \alpha_m \cdot h_m(x) \right)$$

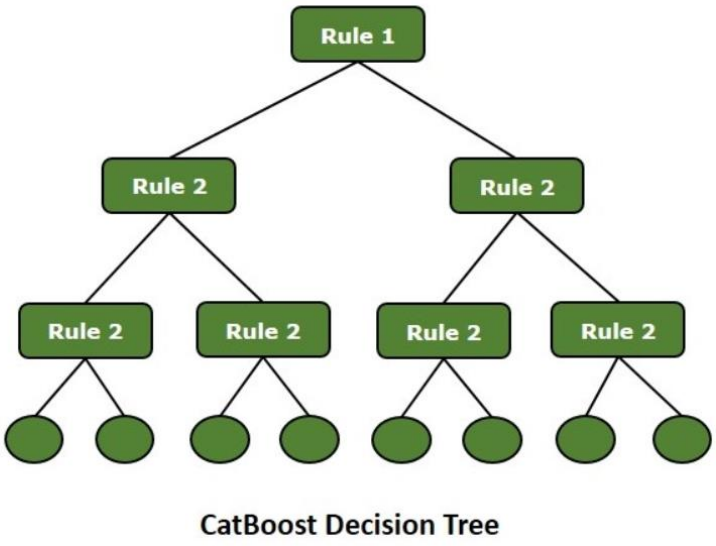
- Ưu điểm:
  - Dễ triển khai và có thể kết hợp với nhiều loại mô hình yếu khác nhau.
  - Tăng hiệu suất đáng kể so với một mô hình đơn lẻ.
  - Hoạt động tốt trên dữ liệu sạch, ít nhiễu.
- Nhược điểm:
  - Nhạy cảm với dữ liệu nhiễu và outliers vì các mẫu khó dự đoán được tăng trọng số liên tục.
  - Có thể overfitting nếu số vòng lặp quá nhiều.

- Tốc độ huấn luyện chậm hơn so với các mô hình bagging (như Random Forest) khi số vòng lớn.

c. Ứng dụng trong phát hiện gian lận thẻ tín dụng.

- AdaBoost có thể phát hiện các mẫu gian lận khó nhận diện bằng cách tập trung nhiều hơn vào các giao dịch bị phân loại sai ở các vòng trước.
- Thường được dùng như một mô hình tham chiếu (baseline) để so sánh với các thuật toán boosting hiện đại như XGBoost, LightGBM, CatBoost.
- Hiệu quả hơn khi kết hợp với các kỹ thuật xử lý mất cân bằng dữ liệu như SMOTE hoặc điều chỉnh trọng số lớp (class weight).

2.4.3: Thuật toán CatBoost:



Hình 2.7: Thuật toán CatBoost

a. Nguyên lý.

CatBoost (Categorical Boosting) là thuật toán gradient boosting được tối ưu đặc biệt cho dữ liệu chứa nhiều biến phân loại (categorical features).

- Khác với XGBoost hoặc LightGBM, CatBoost tự động xử lý biến phân loại bằng target statistics thay vì yêu cầu người dùng mã hóa trước (one-hot encoding).

- Sử dụng cơ chế Ordered Boosting: Khi tính giá trị trung bình của biến phân loại, CatBoost chỉ dùng dữ liệu của các mẫu trước đó trong thứ tự ngẫu nhiên, tránh việc sử dụng thông tin tương lai (target leakage).
- Xây dựng cây quyết định tuần tự, mỗi cây mới học dựa trên phần dư (residuals) của mô hình hiện tại để cải thiện dự đoán.

b. Công thức.

CatBoost tuân theo cơ chế gradient boosting tương tự XGBoost:

- Cập nhật mô hình ở vòng m:

$$F_m(x) = F_{m-1}(x) + \gamma_m \cdot h_m(x)$$

Trong đó:

- $F_m(x)$: mô hình ở vòng m
- $h_m(x)$: cây quyết định mới
- γ_m: hệ số bước (*learning rate*)

- Xử lý biến phân loại:

  Với biến phân loại f và mẫu i, giá trị mã hóa:

$$\text{Enc}_i(f) = \frac{\sum_{j \in \text{prev}(i)} y_j + a \cdot p}{|\text{prev}(i)| + a}$$

Trong đó:

- $\text{prev}(i)$: tập các mẫu trước i trong thứ tự ngẫu nhiên
- y_j: giá trị mục tiêu của mẫu j
- p: giá trị trung bình toàn cục
- a: hệ số làm trơn (*smoothing parameter*)

- Ưu điểm:
  - Xử lý biến phân loại trực tiếp, giảm công đoạn tiền xử lý.
  - Khả năng chống overfitting tốt nhờ cơ chế Ordered Boosting.

- Hoạt động tốt trên cả dữ liệu vừa và nhỏ.
- Ít phải điều chỉnh siêu tham số so với XGBoost/LightGBM.

- Nhược điểm:
  - Huấn luyện chậm hơn LightGBM khi dữ liệu rất lớn.
  - Yêu cầu nhiều bộ nhớ hơn với dữ liệu có nhiều biến phân loại và số lượng mẫu lớn.
  - Cộng đồng người dùng ít hơn XGBoost và LightGBM.

c. Ứng dụng trong phát hiện gian lận thẻ tín dụng.

- Phù hợp khi dữ liệu có nhiều đặc trưng dạng phân loại (ví dụ: loại giao dịch, vị trí, loại thiết bị).
- Tự động mã hóa biến phân loại giúp tiết kiệm thời gian xử lý dữ liệu.
- Có khả năng phát hiện các mẫu gian lận tinh vi nhờ khai thác tốt các mối quan hệ phức tạp giữa biến số.

2.4.4: Thuật toán:



Hình 2.8: Thuật toán XGBoost (Extreme Gradient Boosting)

a. Ngyên lý.

XGBoost là một thuật toán gradient boosting được cải tiến nhằm tối ưu tốc độ, hiệu suất và khả năng chống overfitting.

- Mô hình được xây dựng tuần tự: mỗi cây quyết định mới sẽ học dựa trên phần dư (residuals) của mô hình hiện tại để giảm lỗi dự đoán.
- Sử dụng regularization (L1, L2) trong hàm mục tiêu để kiểm soát độ phức tạp của cây, giúp mô hình tổng quát hóa tốt hơn.
- Hỗ trợ song song hóa quá trình xây cây, tăng tốc huấn luyện.
- Có thể xử lý dữ liệu mất cân bằng thông qua tham số scale_pos_weight.

b. Công thức.

- Hàm mục tiêu tổng quát:

$$\text{Obj} = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k)$$

Trong đó:

- l: hàm mất mát (logistic loss cho phân loại, squared loss cho hồi quy)
- $\Omega(f_k) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$ là **hàm regularization**
- T: số lá của cây
- w_j: giá trị dự đoán tại lá thứ j
- γ, λ: siêu tham số điều chỉnh độ phức tạp

- Cập nhật mô hình ở vòng t:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta \cdot f_t(x_i)$$

- η: learning rate
- f_t: cây quyết định mới được thêm vào

- Ưu điểm:
  - Hiệu suất cao, thường nằm trong nhóm thuật toán chiến thắng các cuộc thi về dữ liệu (Kaggle, KDD Cup).
  - Hỗ trợ regularization mạnh mẽ (L1, L2) giúp chống overfitting.

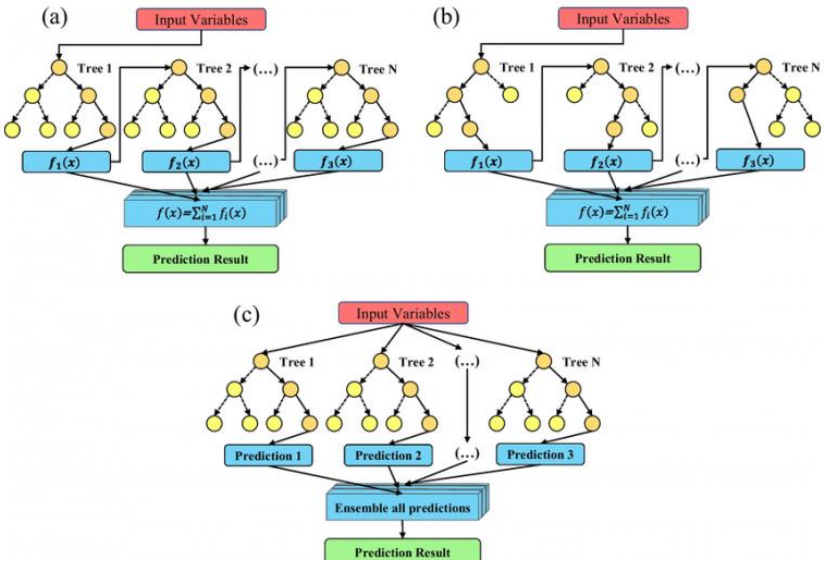
- Linh hoạt trong việc điều chỉnh siêu tham số.
- Hỗ trợ xử lý dữ liệu mất cân bằng thông qua tham số scale_pos_weight.

- Nhược điểm:
  - Huấn luyện chậm hơn LightGBM trên dữ liệu rất lớn.
  - Cần nhiều bộ nhớ khi số cây và độ sâu cây lớn.
  - Yêu cầu tinh chỉnh siêu tham số cẩn thận để đạt hiệu suất tối ưu.

c. Ứng dụng trong phát hiện gian lận thẻ tín dụng.

- XGBoost thường cho độ chính xác (Precision) và khả năng phát hiện gian lận (Recall) cao hơn so với các thuật toán đơn giản.
- Khả năng xử lý tốt dữ liệu mất cân bằng, đặc biệt khi kết hợp với scale_pos_weight hoặc các kỹ thuật tái lấy mẫu (SMOTE, undersampling).
- Được dùng rộng rãi trong các dự án chống gian lận tại ngân hàng, công ty thẻ tín dụng và các nền tảng thương mại điện tử.

2.4.5: Thuật toán LightGBM (Light Gradient Boosting Machine):



Hình 2.9: Thuật toán LightGBM (Light Gradient Boosting Machine)

a. Nguyên lý.

LightGBM là một thuật toán gradient boosting được phát triển bởi Microsoft, tối ưu cho tốc độ và bộ nhớ khi xử lý dữ liệu lớn.

- Chiến lược xây cây: Sử dụng phương pháp leaf-wise growth (tăng trưởng theo lá) thay vì level-wise như các thuật toán boosting truyền thống. Ở mỗi bước, LightGBM mở rộng lá có mức giảm hàm mất mát (loss reduction) lớn nhất, giúp tối ưu hiệu quả học.
- Histogram-based decision tree: Các giá trị đặc trưng được gom thành các nhóm (bin), giúp tăng tốc tính toán và giảm bộ nhớ.
- Hỗ trợ xử lý dữ liệu mất cân bằng thông qua các tham số is_unbalance hoặc scale_pos_weight.

b. Công thức.

Tương tự XGBoost, LightGBM tối ưu hàm mục tiêu tổng quát:

$$\text{Obj} = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k)$$

- l: hàm mất mát (logistic loss cho phân loại)
- $\Omega(f_k)$: thành phần regularization để kiểm soát độ phức tạp của cây

- Cập nhật mô hình ở vòng t:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta \cdot f_t(x_i)$$

- η: learning rate
- f_t: cây quyết định mới được thêm vào

Điểm khác biệt chính là LightGBM chọn lá có mức giảm loss lớn nhất thay vì mở rộng toàn bộ một tầng của cây.

- Ưu điểm:
  - Tốc độ huấn luyện rất nhanh, đặc biệt với dữ liệu hàng triệu mẫu.
  - Tiêu tốn ít bộ nhớ nhờ cơ chế histogram-based.
  - Hỗ trợ tốt dữ liệu mất cân bằng.
  - Có khả năng đạt độ chính xác cao với ít cây hơn so với XGBoost.

- Nhược điểm:
    - Dễ overfitting nếu không giới hạn độ sâu hoặc số lá.
    - Kết quả có thể thay đổi với dữ liệu nhỏ nếu không điều chỉnh tham số cẩn thận.
    - Khó giải thích trực quan so với các mô hình đơn giản.

c. Ứng dụng trong phát hiện gian lận thẻ tín dụng.

- Phù hợp cho hệ thống phát hiện gian lận gần thời gian thực nhờ tốc độ dự đoán nhanh.
- Xử lý tốt dữ liệu mất cân bằng khi dùng is_unbalance = True hoặc scale_pos_weight.
- Được nhiều ngân hàng và công ty tài chính lựa chọn khi cần vừa hiệu suất cao, vừa khả năng mở rộng cho dữ liệu lớn.

2.5: Các chỉ số đánh giá mô hình.

2.5.1: Confusion Matrix (Ma trận nhầm lẫn):

Ma trận nhầm lẫn thể hiện số lượng dự đoán đúng/sai của mô hình cho từng lớp:

	Dự đoán gian lận	Dự đoán hợp lệ
Thực tế gian lận	TP (True Positive)	FN (False Negative)
Thực tế hợp lệ	FP (False Positive)	TN (True Negative)

- TP: Số giao dịch gian lận được dự đoán đúng.
- FP: Số giao dịch hợp lệ bị dự đoán nhầm là gian lận.
- FN: Số giao dịch gian lận bị bỏ sót.
- TN: Số giao dịch hợp lệ được dự đoán đúng.

2.5.2: Accuracy (Độ chính xác):

Tỷ lệ dự đoán đúng trên tổng số mẫu:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- Ưu điểm: Dễ hiểu.
- Nhược điểm: Không phản ánh đúng hiệu quả với dữ liệu mất cân bằng (có thể rất cao ngay cả khi bỏ sót nhiều gian lận).

2.5.3: Precision (Độ chính xác dự đoán dương):

Tỷ lệ dự đoán gian lận đúng trên tổng số dự đoán gian lận:

$$Precision = \frac{TP}{TP + FP}$$

- Giá trị cao ⇒ Ít báo động giả (false positive).
- Quan trọng khi chi phí xử lý báo động giả cao.

2.5.4: Recall (Độ bao phủ / Độ nhạy):

Tỷ lệ phát hiện đúng gian lận trên tổng số gian lận thực tế:

$$Recall = \frac{TP}{TP + FN}$$

- Giá trị cao ⇒ Mô hình ít bỏ sót gian lận.
- Quan trọng khi chi phí bỏ sót gian lận cao (false negative).

2.5.5: F1-score:

Trung bình điều hòa của Precision và Recall:

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

- Giá trị cao ⇒ Mô hình cân bằng tốt giữa Precision và Recall.
- Hữu ích khi dữ liệu mất cân bằng.

2.5.6: ROC Curve và AUC:

- ROC (Receiver Operating Characteristic): Đường cong biểu diễn mối quan hệ giữa True Positive Rate (TPR) và False Positive Rate (FPR) khi thay đổi ngưỡng phân loại.

$$TPR = Recall = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

- AUC (Area Under the Curve): Diện tích dưới đường ROC, giá trị trong khoảng [0,1].
    - AUC = 0.5 ⇒ Mô hình dự đoán ngẫu nhiên.
    - AUC gần 1 ⇒ Mô hình phân biệt tốt giữa hai lớp.

2.5.7: Lý do sử dụng nhiều chỉ số:

Trong phát hiện gian lận thẻ tín dụng, Recall thường được ưu tiên để giảm bỏ sót giao dịch gian lận, nhưng vẫn cần Precision để hạn chế báo động giả. Vì vậy, kết hợp F1-score và AUC giúp đánh giá mô hình toàn diện hơn.

CHƯƠNG 3: DỮ LIỆU VÀ PHƯƠNG PHÁP THỰC HIỆN

3.1. Nguồn dữ liệu.

Bộ dữ liệu được sử dụng trong nghiên cứu là Credit Card Fraud Detection Dataset, được công bố công khai trên nền tảng Kaggle. Tập dữ liệu này được thu thập từ các giao dịch thẻ tín dụng của khách hàng tại châu Âu trong năm 2013, trong khoảng thời gian 2 ngày.

- Quy mô: 284.807 giao dịch.
- Đặc điểm: bao gồm cả giao dịch hợp lệ và giao dịch gian lận.
- Ẩn danh hóa dữ liệu: để bảo mật thông tin người dùng, các biến gốc đã được biến đổi bằng kỹ thuật Phân tích thành phần chính (PCA), tạo thành 28 đặc trưng (V1–V28).
- Mục tiêu nghiên cứu: biến Class là nhãn mục tiêu, với hai giá trị:

  0: giao dịch hợp lệ.

  1: giao dịch gian lận.

Bộ dữ liệu này hiện được xem là chuẩn benchmark cho các nghiên cứu và so sánh thuật toán trong bài toán phát hiện gian lận thẻ tín dụng bằng học máy và trí tuệ nhân tạo.

3.2. Mô tả tập dữ liệu.

Tập dữ liệu sử dụng trong nghiên cứu là Credit Card Fraud Detection Dataset gồm 284.807 giao dịch với 31 thuộc tính. Đây là bộ dữ liệu công khai, thường dùng trong các nghiên cứu về phát hiện gian lận thẻ tín dụng.

- Time: thời gian (tính bằng giây) kể từ giao dịch đầu tiên trong tập dữ liệu.
- V1 → V28: 28 đặc trưng đã được biến đổi ẩn danh bằng kỹ thuật PCA nhằm bảo mật thông tin nhạy cảm.
- Amount: số tiền của giao dịch.
- Class: nhãn mục tiêu, trong đó:

  0: giao dịch hợp lệ (không gian lận).

1: giao dịch gian lận.

- Đặc điểm quan trọng của dữ liệu:
  - Dữ liệu mất cân bằng nghiêm trọng: chỉ có 492 giao dịch gian lận trên tổng số 284.807 giao dịch (tương ứng 0,1727%).
  - Không có giá trị bị thiếu (missing value).
  - Các thuộc tính V1–V28 đã được chuẩn hóa sẵn, trong khi Amount có phân bố lệch phải mạnh.

```
[9] data_df.head()
```

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	V11	V1
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	0.090794	-0.551600	-0.61780
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	-0.166974	1.612727	1.06523
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	0.207643	0.624501	0.06608
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	-0.054952	-0.226487	0.17822
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	0.753074	-0.822843	0.53819

Hình 3.1: Đọc và kiểm tra nhanh dữ liệu

- Hiển thị 5 bản ghi đầu tiên của tập dữ liệu, nhằm quan sát sơ bộ cấu trúc và giá trị mẫu của các thuộc tính trước khi tiến hành tiền xử lý và huấn luyện mô hình.

3.3. Thiết lập môi trường và chuẩn bị dữ liệu.

3.3.1. Nền tảng thực hiện:

Toàn bộ quá trình xử lý và huấn luyện mô hình được thực hiện trên nền tảng Google Colab – một công cụ tính toán đám mây do Google cung cấp, hỗ trợ GPU,

thuận tiện cho xử lý dữ liệu và huấn luyện mô hình học sâu. Bên cạnh đó, người dùng có thể sử dụng Jupyter Notebook nếu làm việc trên máy tính cá nhân

a. Ngôn ngữ và thực hiện.

- Ngôn ngữ: Python 3.x
- Thư viện sử dụng:
  - Pandas, Numpy: xử lý, thống kê và thao tác dữ liệu dạng bảng.
  - Matplotlib, Plotly: trực quan hóa dữ liệu và biểu đồ kết quả.
  - Scikit-learn: chuẩn hóa dữ liệu (MinMaxScaler), phân chia tập huấn

luyện/kiểm tra, tính toán độ chính xác mô hình.

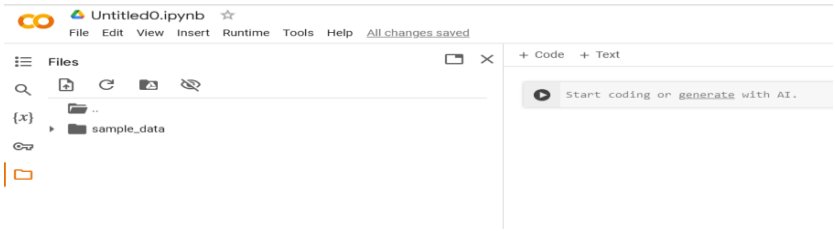
- TensorFlow/Keras: xây dựng và huấn luyện mô hình LSTM.
- Callback: hỗ trợ theo dõi quá trình huấn luyện và lưu mô hình tốt nhất

b. Cấu Hình Phần Cứng (Trong Colab).

- GPU: Tùy chọn GPU (Tesla T4 hoặc P100).
- RAM: khoảng 12.7 GB.
- Bộ xử lý: môi trường máy ảo của Google.

3.3.2. Môi trường thực nghiệm:

Google Colab: Colaboratory, hay " colab " là nền tảng trực tuyến miễn phí của Google hỗ trợ viết mã Python và sử dụng GPU/TPU miễn phí. Người dùng có thể cài đặt các thư viện như TensorFlow, PyTorch, NumPy để huấn luyện mô hình AI, xử lý dữ liệu lớn hoặc trực quan hóa kết quả.



Hình 3.2: Giao diện chính của colab

3.3.3. Triển khai mô hình:

- Nhằm chuẩn bị môi trường, import các thư viện, cấu hình tham số mô hình và chỉ định đường dẫn dữ liệu, để sẵn sàng cho các bước tiếp theo (tiền xử lý dữ liệu, huấn luyện và đánh giá mô hình).

```
pd.set_option('display.max_columns', 100)
# Tham số cho Random Forest
RFC_METRIC = 'gini'  # Chỉ số đánh giá (metric) dùng cho mô hình phân loại Random Forest
NUM_ESTIMATORS = 100 # Số lượng cây quyết định (decision trees) được sử dụng trong RandomForestClassifier
NO_JOBS = 4 # Số lượng tác vụ chạy song song (parallel jobs) trong RandomForestClassifier
# Kiểm định mô hình (Validation)
VALID_SIZE = 0.20 # Xác thực đơn giản (simple validation) bằng train_test_split
TEST_SIZE = 0.20 # Kích thước tập kiểm tra (test set) khi dùng train_test_split
# Kiểm định chéo (Cross-validation)
NUMBER_KFOLDS = 5 # Số lượng phần chia (folds) cho cross-validation
RANDOM_STATE = 2018 # Hạt giống ngẫu nhiên (random seed) để kết quả tái lập
# Tham số cho LightGBM
MAX_ROUNDS = 1000 # Số vòng lặp (iterations) tối đa cho LightGBM
EARLY_STOP = 50 # Dừng sớm (early stopping) trong LightGBM nếu không cải thiện
OPT_ROUNDS = 1000  # Sẽ được điều chỉnh dựa trên số vòng xác thực tốt nhất
VERBOSE_EVAL = 50 # In kết quả chỉ số đánh giá (metric) sau mỗi 50 vòng
IS_LOCAL = False
import os
if(IS_LOCAL):
    PATH="/content/creditcard.csv"
else:
    PATH="/content/creditcard.csv"
```

Hình 3.3: Chỉ định đường dẫn dữ liệu.

3.3.4. Xem tổng quan dữ liệu:

```
data_df.describe()
```

	Time	V1	V2	V3	V4	V5	V6	V7	
count	284807.000000	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848
mean	94813.859575	1.168375e-15	3.416908e-16	-1.379537e-15	2.074095e-15	9.604066e-16	1.487313e-15	-5.556467e-16	1.213
std	47488.145955	1.958696e+00	1.651309e+00	1.516255e+00	1.415869e+00	1.380247e+00	1.332271e+00	1.237094e+00	1.194
min	0.000000	-5.640751e+01	-7.271573e+01	-4.832559e+01	-5.683171e+00	-1.137433e+02	-2.616051e+01	-4.355724e+01	-7.321
25%	54201.500000	-9.203734e-01	-5.985499e-01	-8.903648e-01	-8.486401e-01	-6.915971e-01	-7.682956e-01	-5.540759e-01	-2.086
50%	84692.000000	1.810880e-02	6.548556e-02	1.798463e-01	-1.984653e-02	-5.433583e-02	-2.741871e-01	4.010308e-02	2.235
75%	139320.500000	1.315642e+00	8.037239e-01	1.027196e+00	7.433413e-01	6.119264e-01	3.985649e-01	5.704361e-01	3.273
max	172792.000000	2.454930e+00	2.205773e+01	9.382558e+00	1.687534e+01	3.480167e+01	7.330163e+01	1.205895e+02	2.000

Hình 3.4: Tính toán các thông kê cơ bản.

- Thống kê mô tả cơ bản của dữ liệu, bao gồm số lượng mẫu, giá trị trung bình, độ lệch chuẩn, giá trị nhỏ nhất, lớn nhất và các phân vị. Việc này giúp hiểu rõ đặc điểm phân bố dữ liệu và phát hiện những bất thường (outliers) ngay từ bước khám phá dữ liệu ban đầu.

3.3.5. Kiểm tra dữ liệu bị thiếu và mất cân bằng dữ liệu:

- Kiểm tra và thống kê dữ liệu bị khuyết trong tập dữ liệu. Cụ thể, chương trình tính toán số lượng giá trị khuyết (Total) và tỉ lệ phần trăm giá trị khuyết trên tổng số quan sát (Percent) cho từng thuộc tính, sau đó hiển thị kết quả dưới dạng bảng. Việc này giúp phát hiện các cột có dữ liệu thiếu nhiều, từ đó lựa chọn phương pháp xử lý phù hợp (loại bỏ, thay thế bằng trung bình/trung vị, hoặc sử dụng mô hình dự đoán giá trị thiếu).

```
[11] total = data_df.isnull().sum().sort_values(ascending = False)
     percent = (data_df.isnull().sum()/data_df.isnull().count()*100).sort_values(ascending = False)
     pd.concat([total, percent], axis=1, keys=['Total', 'Percent']).transpose()
```

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	V11	V12	V13	V14	V15	V16	V17	V18	V19	V20	V21	V22	V23	V24
Total	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Percent	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

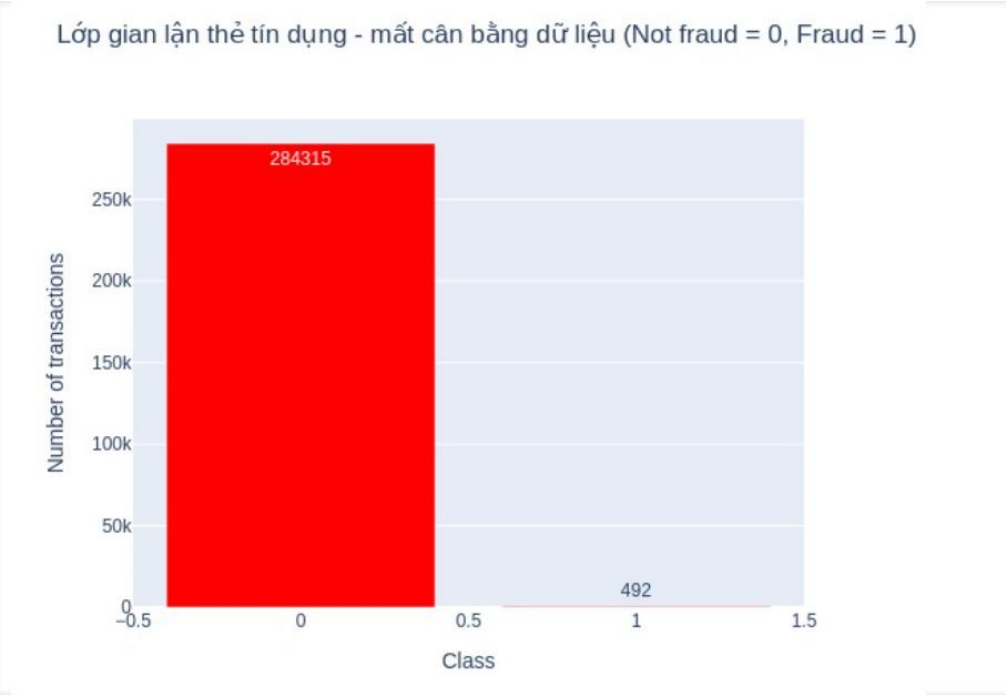
Hình 3.5: Kiểm tra dữ liệu bị khuyết (missing values).

```python
from google.colab import files
import plotly.io as pio

temp = data_df["Class"].value_counts()
df = pd.DataFrame({'Class': temp.index,'values': temp.values})

trace = go.Bar(
    x = df['Class'],y = df['values'],
    name="Credit Card Fraud Class - data unbalance (Not fraud = 0, Fraud = 1)",
    marker=dict(color="Red"),
    text=df['values']
)
data = [trace]
layout = dict(title = 'Lớp gian lận thẻ tín dụng - mất cân bằng dữ liệu (Not fraud = 0, Fraud = 1)',
          xaxis = dict(title = 'Class', showticklabels=True),
          yaxis = dict(title = 'Number of transactions'),
          hovermode = 'closest',width=600
         )
fig = dict(data=data, layout=layout)
iplot(fig, filename='class')

# Lưu hình ảnh dưới dạng tệp JPEG
pio.write_image(fig, 'Lớp gian lận thẻ tín dụng.jpeg')
```



Hình 3.6: Biểu đồ lớp gian lận tín dụng

- Biểu đồ trên thể hiện sự phân bố dữ liệu giữa hai lớp giao dịch: hợp lệ (class = 0) và gian lận (class = 1). Kết quả cho thấy sự mất cân bằng dữ liệu rõ rệt với 284.315 giao dịch hợp lệ so với chỉ 492 giao dịch gian lận. Đây là đặc điểm quan trọng của bài toán phát hiện gian lận thẻ tín dụng, bởi nếu không xử lý, mô hình học máy sẽ dễ thiên lệch về lớp đa số. Do đó, cần áp dụng các kỹ thuật như resampling, SMOTE hoặc gán trọng số lớp để cải thiện khả năng phát hiện gian lận.

CHƯƠNG 4: KHÁM PHÁ VÀ PHÂN TÍCH DỮ LIỆU

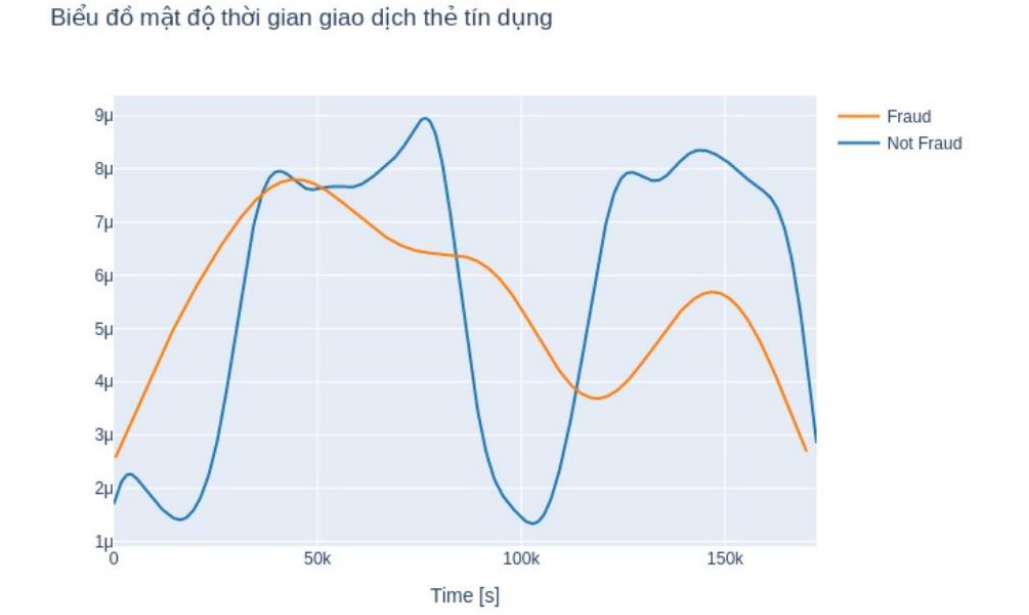
4.1. Các biểu đồ giao dịch theo thời gian.

```
class_0 = data_df.loc[data_df['Class'] == 0]["Time"]
class_1 = data_df.loc[data_df['Class'] == 1]["Time"]

hist_data = [class_0, class_1]
group_labels = ['Not Fraud', 'Fraud']

fig = ff.create_distplot(hist_data, group_labels, show_hist=False, show_rug=False)
fig['layout'].update(title='Biểu đồ mật độ thời gian giao dịch thẻ tín dụng', xaxis=dict(title='Time [s]'))
iplot(fig, filename='dist_only')

# Lưu hình ảnh dưới dạng tệp JPEG
pio.write_image(fig, 'Biểu đồ mật độ thời gian giao dịch thẻ tín dụng.jpeg')
```



Hình 4.1: Biểu đồ mật độ phân bố thời gian giao dịch

- Qua biểu đồ mật độ thời gian giao dịch, ta nhận thấy các giao dịch hợp lệ (Not Fraud) có sự phân bố rõ rệt thành nhiều đỉnh, đặc biệt tập trung nhiều ở giai đoạn cuối, khoảng 30.000 giây. Trong khi đó, các giao dịch gian lận (Fraud) lại có xu hướng phân bố trải đều hơn, ít biến động, không tập trung vào một khoảng thời gian cụ thể nào. Điều này cho thấy đặc trưng Time có thể được sử dụng để hỗ trợ việc phân biệt giữa giao dịch hợp lệ và gian lận. Tuy nhiên, vì vẫn tồn tại sự chồng lấn giữa hai nhóm nên đặc trưng này chưa đủ mạnh để tách biệt hoàn toàn hai lớp dữ liệu.

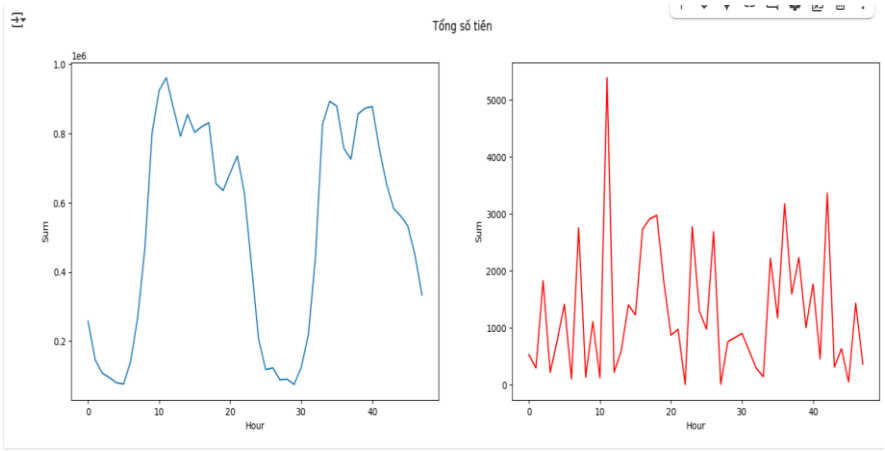
```
data_df['Hour'] = data_df['Time'].apply(lambda x: np.floor(x / 3600))
tmp = data_df.groupby(['Hour', 'Class'])['Amount'].aggregate(['min', 'max', 'count', 'sum', 'mean', 'median', 'var']).reset_index()
df = pd.DataFrame(tmp)
df.columns = ['Hour', 'Class', 'Min', 'Max', 'Transactions', 'Sum', 'Mean', 'Median', 'Var']
df.head()
```

	Hour	Class	Min	Max	Transactions	Sum	Mean	Median	Var
0	0.0	0	0.0	7712.43	3961	256572.87	64.774772	12.990	45615.821201
1	0.0	1	0.0	529.00	2	529.00	264.500000	264.500	139920.500000
2	1.0	0	0.0	1769.69	2215	145806.76	65.826980	22.820	20053.615770
3	1.0	1	59.0	239.93	2	298.93	149.465000	149.465	16367.832450
4	2.0	0	0.0	4002.88	1555	106989.39	68.803466	17.900	45355.430437

Các bước tiếp theo: Tạo mã bằng df | Xem các đồ thị được đề xuất | New interactive sheet

Hình 4.2: Phân biệt giữa giao dịch thường và gian lận

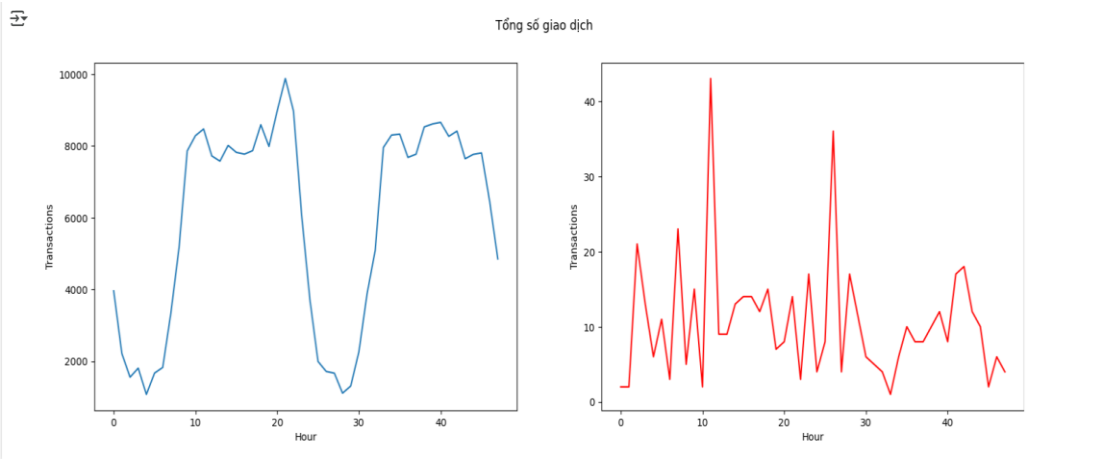
```
fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(18,6))
s = sns.lineplot(ax = ax1, x="Hour", y="Sum", data=df.loc[df.Class==0])
s = sns.lineplot(ax = ax2, x="Hour", y="Sum", data=df.loc[df.Class==1], color="red")
plt.suptitle("Tổng số tiền")
plt.show();
```



Hình 4.3: Biểu đồ thể hiện tổng số tiền giao dịch theo từng giờ

- Biểu đồ cho thấy giao dịch hợp lệ (Class = 0) có tổng số tiền rất lớn, biến động theo quy luật rõ ràng ở một số khoảng giờ nhất định. Trong khi đó, giao dịch gian lận (Class = 1) có tổng số tiền nhỏ hơn nhiều nhưng dao động thất thường và xuất hiện các đỉnh bất thường tại một số giờ. Điều này phản ánh tính bất quy tắc của gian lận, là cơ sở quan trọng để phát hiện giao dịch đáng ngờ.

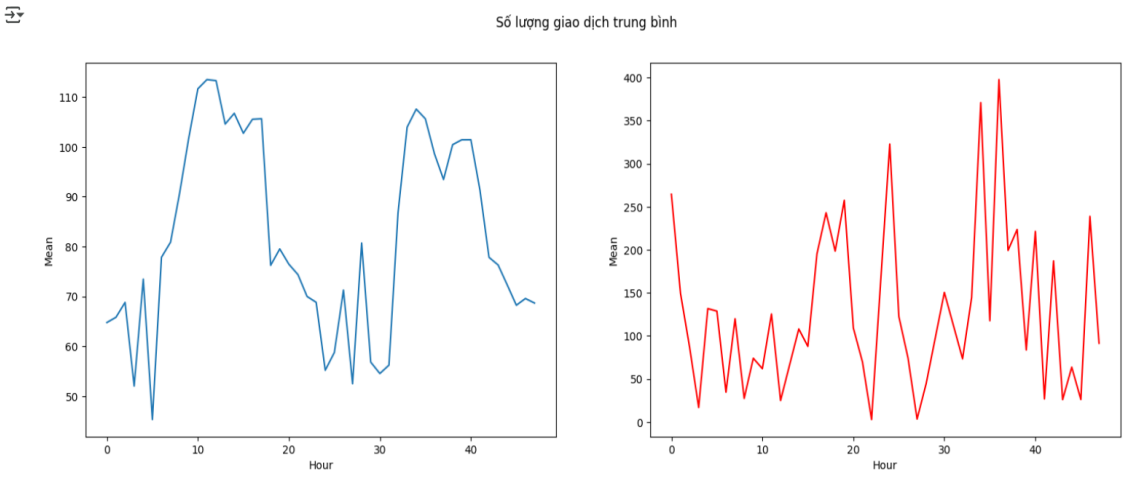
```
[15] fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(18,6))
     s = sns.lineplot(ax = ax1, x="Hour", y="Transactions", data=df.loc[df.Class==0])
     s = sns.lineplot(ax = ax2, x="Hour", y="Transactions", data=df.loc[df.Class==1], color="red")
     plt.suptitle("Tổng số giao dịch")
     plt.show();
```



Hình 4.4: Biểu đồ thể hiện tổng số giao dịch

- Giao dịch hợp lệ (Class = 0) có số lượng lớn, phân bố theo quy luật rõ rệt theo từng giờ. Ngược lại, giao dịch gian lận (Class = 1) có số lượng rất ít, dao động thất thường và xuất hiện một số giờ tăng đột biến, thể hiện tính bất thường.

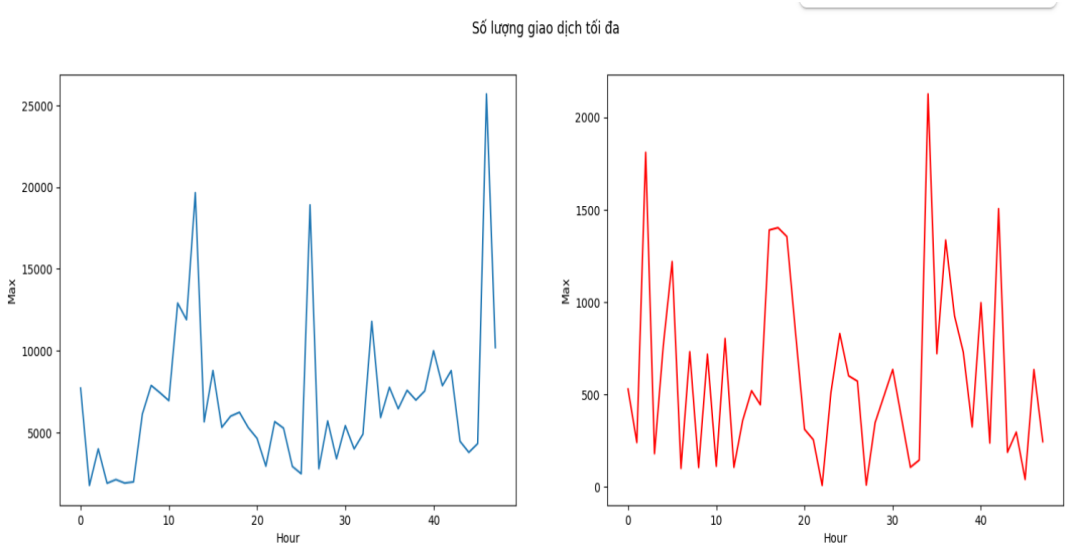
```
fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(18,6))
s = sns.lineplot(ax = ax1, x="Hour", y="Mean", data=df.loc[df.Class==0])
s = sns.lineplot(ax = ax2, x="Hour", y="Mean", data=df.loc[df.Class==1], color="red")
plt.suptitle("Số lượng giao dịch trung bình")
plt.show();
```



Hình 4.5: Biểu đồ thể hiện số lượng giao dịch

- Giao dịch hợp lệ (Class = 0) có giá trị trung bình ổn định theo giờ, dao động quanh mức 70–110. Trong khi đó, giao dịch gian lận (Class = 1) ít hơn nhưng giá trị trung bình biến động mạnh và xuất hiện nhiều đỉnh bất thường, phản ánh tính không ổn định của các giao dịch gian lận.

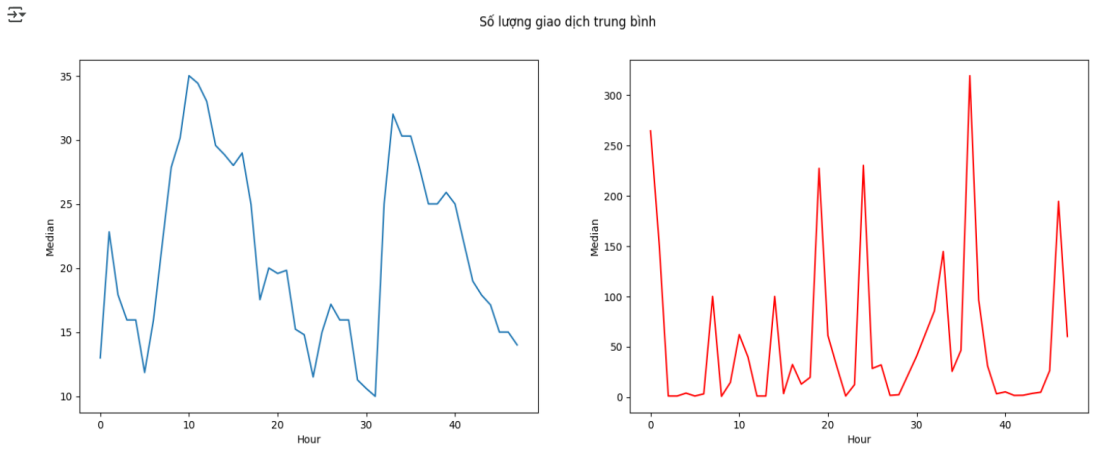
```
fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(18,6))
s = sns.lineplot(ax = ax1, x="Hour", y="Max", data=df.loc[df.Class==0])
s = sns.lineplot(ax = ax2, x="Hour", y="Max", data=df.loc[df.Class==1], color="red")
plt.suptitle("Số lượng giao dịch tối đa")
plt.show();
```



Hình 4.6: Biểu đồ thể hiện số lượng giao dịch tối đa

- Biểu đồ bên trái (màu xanh) thể hiện phân bố số lượng giao dịch tối đa theo từng giờ trong ngày đối với các giao dịch hợp lệ, trong khi biểu đồ bên phải (màu đỏ) biểu diễn số lượng giao dịch tối đa theo giờ đối với các giao dịch gian lận. Việc đặt hai biểu đồ cạnh nhau giúp dễ dàng quan sát và so sánh trực quan sự khác biệt trong hành vi giao dịch giữa hai nhóm này theo yếu tố thời gian.

```
[18] fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(18,6))
     s = sns.lineplot(ax = ax1, x="Hour", y="Median", data=df.loc[df.Class==0])
     s = sns.lineplot(ax = ax2, x="Hour", y="Median", data=df.loc[df.Class==1], color="red")
     plt.suptitle("Số lượng giao dịch trung bình")
     plt.show();
```

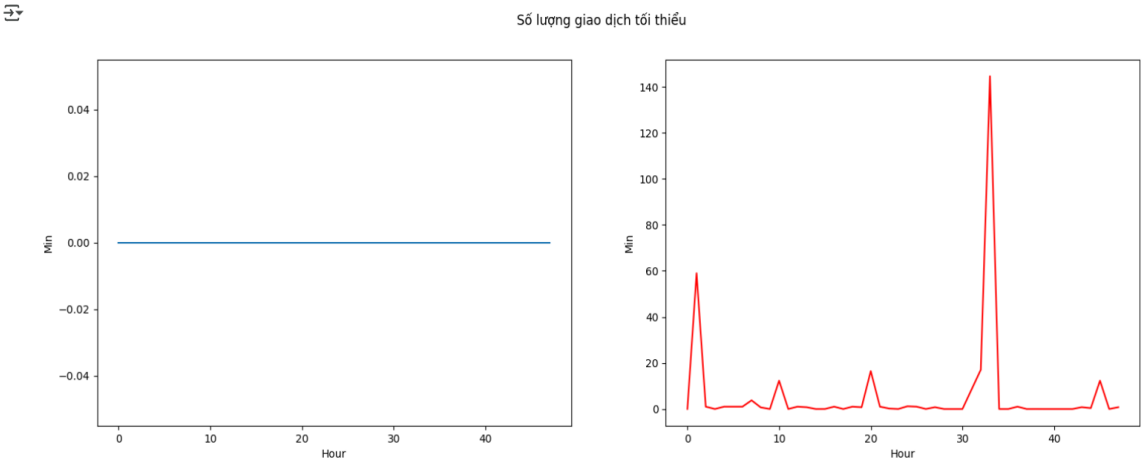


Hình 4.7: Biểu đồ thể hiện số lượng giao dịch trung bình

- Biểu đồ bên trái (màu xanh) thể hiện phân bố số lượng giao dịch trung bình theo từng giờ trong ngày đối với các giao dịch hợp lệ, trong khi biểu đồ bên phải (màu đỏ) biểu diễn số lượng giao dịch trung bình theo giờ đối với các giao dịch gian lận. Việc đặt hai biểu đồ cạnh nhau cho phép so sánh trực quan, từ đó dễ dàng nhận thấy sự khác biệt trong hành vi giao dịch của hai nhóm này dựa trên yếu tố thời gian.

```
[19] fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(18,6))
     s = sns.lineplot(ax = ax1, x="Hour", y="Min", data=df.loc[df.Class==0])
     s = sns.lineplot(ax = ax2, x="Hour", y="Min", data=df.loc[df.Class==1], color="red")
     plt.suptitle("Số lượng giao dịch tối thiểu")
     plt.show();
```

0 giây

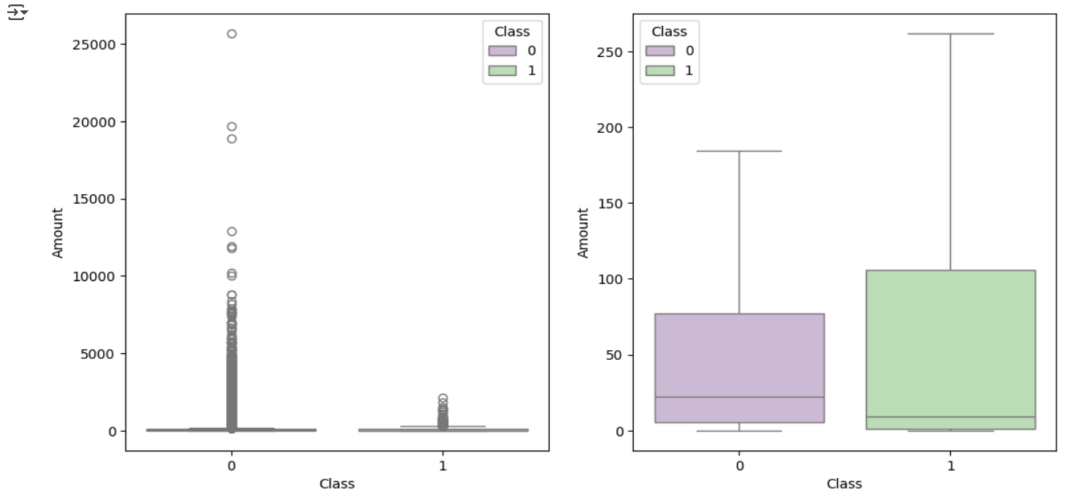


Hình: 4.8: Biểu đồ thể hiện số lượng giao dịch tối thiểu

- Biểu đồ bên trái (màu xanh) thể hiện số lượng giao dịch tối thiểu theo từng giờ trong ngày đối với các giao dịch hợp lệ, trong đó giá trị luôn bằng 0 nên đường biểu diễn nằm ngang. Biểu đồ bên phải (màu đỏ) biểu diễn số lượng giao dịch tối thiểu theo giờ đối với các giao dịch gian lận, cho thấy có sự dao động đáng kể và xuất hiện các đỉnh cao bất thường ở một số thời điểm. Việc đặt hai biểu đồ cạnh nhau giúp làm nổi bật sự khác biệt rõ rệt giữa giao dịch hợp lệ và giao dịch gian lận ở khía cạnh số lượng giao dịch tối thiểu.

4.2. Giá trị của các giao dịch.

```
[20] fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(12,6))
     s = sns.boxplot(ax = ax1, x="Class", y="Amount", hue="Class",data=data_df,
                     palette="PRGn",showfliers=True)
     s = sns.boxplot(ax = ax2, x="Class", y="Amount", hue="Class",data=data_df,
                     palette="PRGn",showfliers=False)
     plt.show();
```



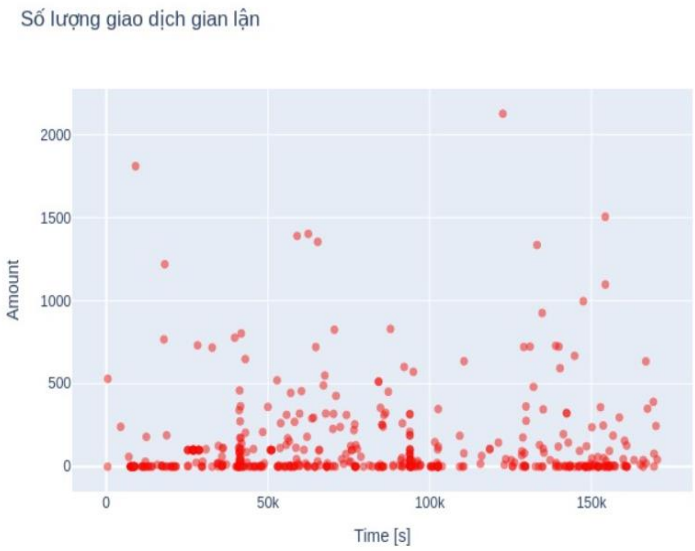
Hình 4.9: Biểu đồ hộp so sánh số tiền giao dịch giữa giao dịch hợp lệ và giao dịch gian lận

- Biểu đồ hộp cho thấy sự phân bố số tiền giao dịch giữa hai nhóm: hợp lệ (Class = 0) và gian lận (Class = 1). Ở biểu đồ bên trái (có outliers), nhiều giao dịch hợp lệ có giá trị tiền rất lớn, trong khi giao dịch gian lận cũng xuất hiện một số ngoại lệ nhưng ít hơn. Biểu đồ bên phải (loại bỏ outliers) cho thấy số tiền trung vị của giao dịch gian lận cao hơn giao dịch hợp lệ, đồng thời khoảng biến động cũng rộng hơn, phản ánh sự chênh lệch rõ rệt về giá trị giao dịch giữa hai nhóm.

```
[23] fraud = data_df.loc[data_df['Class'] == 1]

     trace = go.Scatter(
         x = fraud['Time'],y = fraud['Amount'],
         name="Amount",
          marker=dict(
                     color='rgb(238,23,11)',
                     line=dict(
                         color='red',
                         width=1),
                     opacity=0.5,
                    ),
         text= fraud['Amount'],
         mode = "markers"
     )
     data = [trace]
     layout = dict(title = 'Số lượng giao dịch gian lận',
               xaxis = dict(title = 'Time [s]', showticklabels=True),
               yaxis = dict(title = 'Amount'),
               hovermode='closest'
              )
     fig = dict(data=data, layout=layout)
     iplot(fig, filename='fraud-amount')

     # Save the figure as a JPEG file
     pio.write_image(fig, 'Số lượng giao dịch gian lận.jpeg')
```



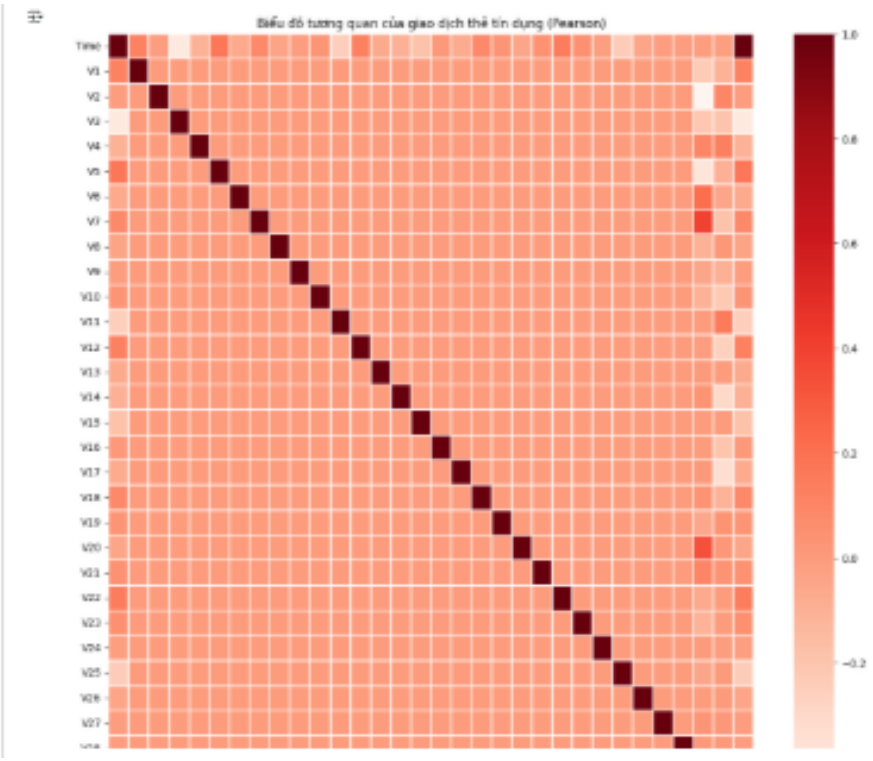
Hình 4.10: Biểu đồ phân tán (Scatter Plot)

Phần lớn giao dịch gian lận có giá trị nhỏ (sát trục hoành).

- Một số điểm rời rạc với giá trị rất cao (từ 500 đến trên 2000), thể hiện các giao dịch gian lận có số tiền lớn bất thường.
- Giao dịch gian lận phân bố rải rác theo thời gian, không tập trung vào một giai đoạn nhất định → gian lận có thể xảy ra bất kỳ lúc nào.

4.3. Sự tương quan giữa các đặc trưng.

```
plt.figure(figsize = (14,14))
plt.title('Biểu đồ tương quan của giao dịch thẻ tín dụng (Pearson)')
corr = data_df.corr()
sns.heatmap(corr,xticklabels=corr.columns,yticklabels=corr.columns,linewidths=.1,cmap="Reds")
plt.show()
```



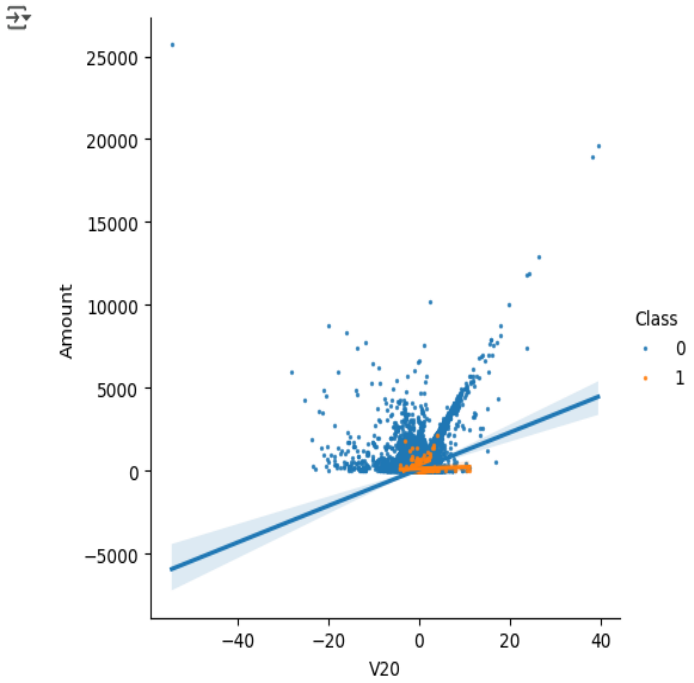
Hình 4.11: Ma trận tương quan (Pearson) giữa các thuộc tính trong bộ dữ liệu giao dịch thẻ tín dụng

- Biểu đồ heatmap ma trận tương quan (Pearson) cho thấy các thuộc tính trong bộ dữ liệu giao dịch thẻ tín dụng phần lớn có mối tương quan yếu với nhau, thể hiện qua các ô màu nhạt. Một số cặp thuộc tính có mức tương quan mạnh hơn, biểu diễn bằng các ô màu đỏ đậm, cho thấy khả năng tồn tại mối liên hệ chặt chẽ giữa chúng. Đường chéo chính của biểu đồ có giá trị tương quan tuyệt đối (1.0) do mỗi biến hoàn toàn tương quan với chính nó. Điều này giúp ta nhận diện nhanh các thuộc tính có thể liên quan hoặc trùng lặp thông tin, hỗ trợ cho quá trình chọn lọc và phân tích đặc trưng trong các bước tiếp theo.

4.3.1. Biểu đồ hồi quy giữa các biến và Amount:

- Biểu đồ hồi quy cho biến V20 có mối quan hệ tuyến tính với Amount. Phần lớn các điểm dữ liệu tập trung quanh gốc tọa độ, trong đó giao dịch bình thường (Class 0) chiếm đa số và trải rộng hơn, còn giao dịch gian lận (Class 1) tập trung ở mức Amount thấp. Điều này gợi ý V20 có thể hỗ trợ phân biệt hai lớp nhưng chưa rõ ràng mạnh.

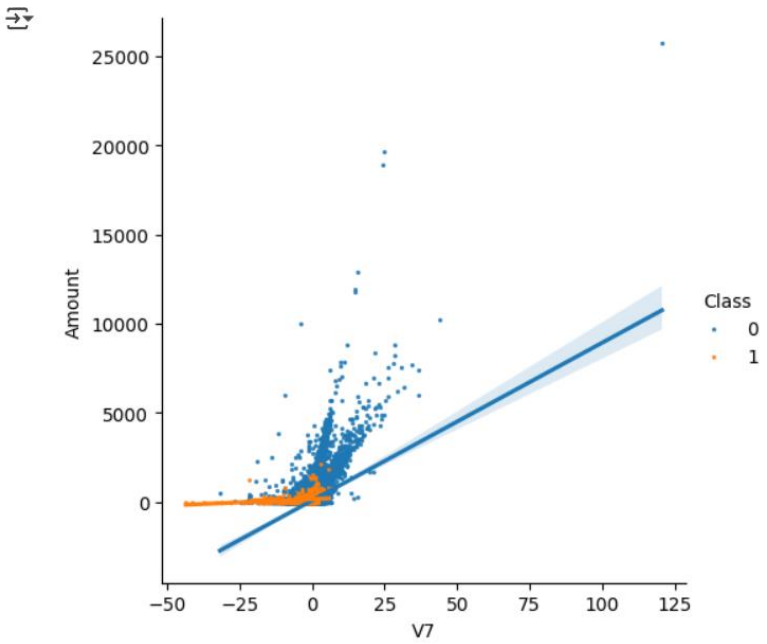
```
[25] s = sns.lmplot(x='V20', y='Amount',data=data_df, hue='Class',
                    fit_reg=True,scatter_kws={'s':2})
     plt.show()
```



Hình 4.12: Biểu đồ hồi quy biến giữa V20 và Amount

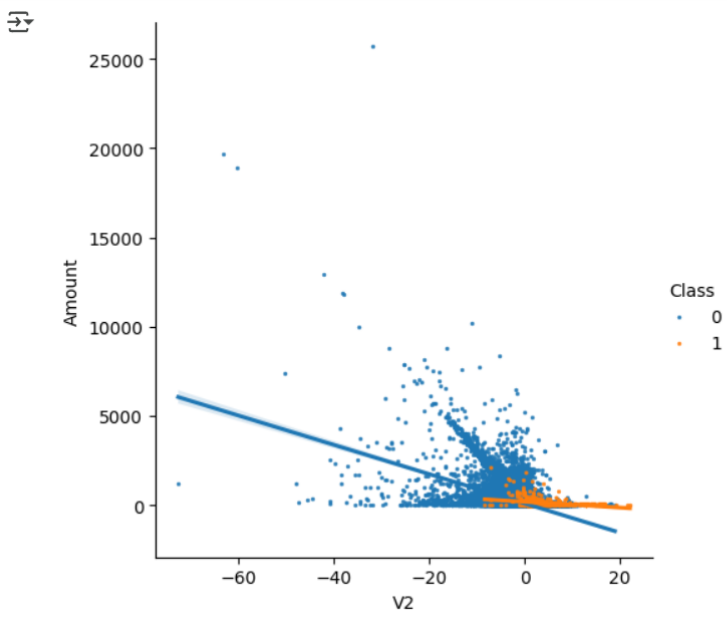
- Biểu đồ hồi quy giữa V7 và Amount cho thấy phần lớn điểm dữ liệu tập trung quanh gốc tọa độ. Các giao dịch bình thường (Class 0) chiếm đa số và phân tán rộng với một số điểm ngoại lai có giá trị Amount rất cao, trong khi giao dịch gian lận (Class 1) chủ yếu tập trung ở mức Amount thấp gần 0. Điều này cho thấy V7 có xu hướng tuyến tính với Amount, nhưng khả năng phân biệt hai lớp còn hạn chế.

```
s = sns.lmplot(x='V7', y='Amount',data=data_df, hue='Class',
               fit_reg=True,scatter_kws={'s':2})
plt.show()
```



Hình 4.13: Biểu đồ hồi quy giữa V7 và Amount

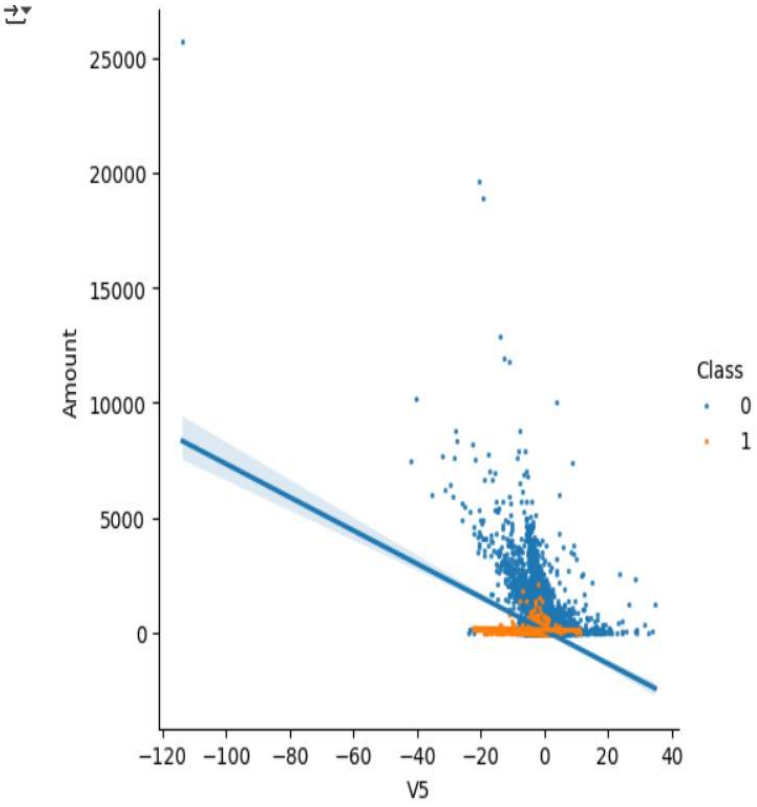
```
s = sns.lmplot(x='V2', y='Amount',data=data_df, hue='Class', fit_reg=True,scatter_kws={'s':2})
plt.show()
```



Hình 4.14: Biểu đồ hồi quy giữa V2 và Amount

- Biểu đồ cho thấy phần lớn giao dịch tập trung quanh giá trị nhỏ của V2 và Amount, trong đó Class 0 chiếm đa số và trải rộng hơn, còn Class 1 chủ yếu nằm ở khu vực Amount thấp gần 0. Mối quan hệ tuyến tính tồn tại nhưng không rõ rệt, cho thấy V2 không phải là biến phân biệt mạnh giữa hai lớp.

```
[28] s = sns.lmplot(x='V5', y='Amount',data=data_df, hue='Class', fit_reg=True,scatter_kws={'s':2})
     plt.show()
```



Hình 4.15: Biểu đồ hồi quy giữa V5 và Amount

- Hầu hết các điểm dữ liệu tập trung quanh gốc tọa độ với Amount nhỏ. Giao dịch Class 0 phân bố rải rác và trải rộng theo cả chiều dương lẫn âm của V5, trong khi Class 1 chủ yếu tập trung ở vùng Amount thấp gần 0. Mối quan hệ tuyến tính giữa V5 và Amount không rõ rệt, cho thấy biến này khó phân biệt gian lận và giao dịch hợp lệ.

4.3.2. Biểu đồ mật độ phân bố đặc trưng:

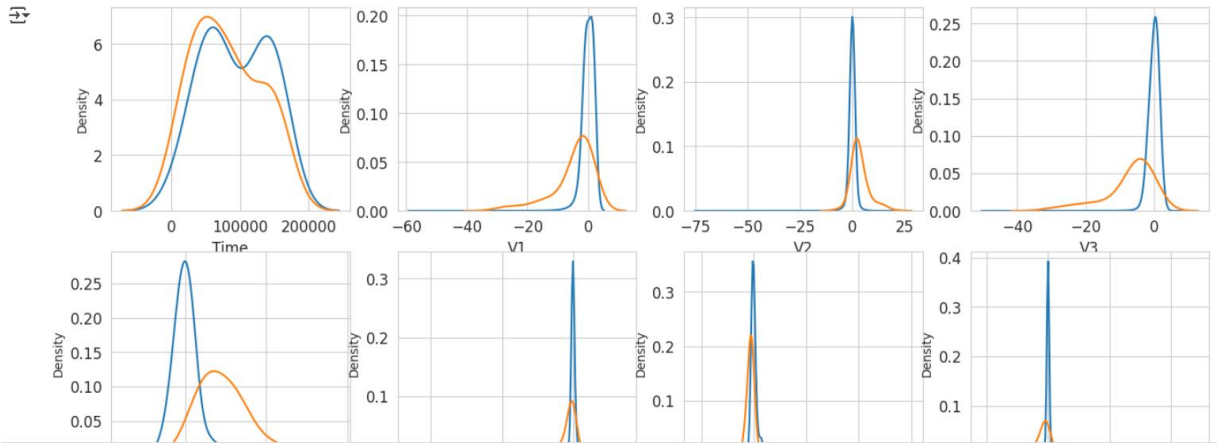
- Để quan sát sự khác biệt về phân phối dữ liệu giữa giao dịch hợp lệ (Class 0) và gian lận (Class 1), ta sử dụng biểu đồ mật độ (KDE) cho từng biến trong tập dữ liệu

```
var = data_df.columns.values

i = 0
t0 = data_df.loc[data_df['Class'] == 0]
t1 = data_df.loc[data_df['Class'] == 1]

sns.set_style('whitegrid')
plt.figure()
fig, ax = plt.subplots(8,4,figsize=(16,28))

for feature in var:
    i += 1
    plt.subplot(8,4,i)
    sns.kdeplot(t0[feature], bw=0.5,label="Class = 0")
    sns.kdeplot(t1[feature], bw=0.5,label="Class = 1")
    plt.xlabel(feature, fontsize=12)
    locs, labels = plt.xticks()
    plt.tick_params(axis='both', which='major', labelsize=12)
plt.show();
```



Hình 4.16: Biểu đồ mật độ phân phối (KDE) các biến theo Class

- Phần lớn các biến cho thấy phân phối của Class 0 và Class 1 khác nhau rõ rệt, đặc biệt ở một số biến như V2, V4, V10, V12, V14, V17, V20 nơi các đỉnh phân phối lệch hẳn nhau. Điều này chứng tỏ các biến đó có giá trị trong việc phân biệt giao dịch gian lận và hợp lệ. Một số biến khác thì phân phối của hai lớp gần trùng nhau, ít khả năng phân tách.

CHƯƠNG 5: XÂY DỰNG CÁC MÔ HÌNH DỰ ĐOÁN

5.1. Chuẩn bị dữ liệu cho mô hình.

5.1.1. Xác định các biến dự báo và biến mục tiêu:

Trước khi xây dựng mô hình dự đoán, bước đầu tiên quan trọng là xác định rõ ràng biến mục tiêu và các biến độc lập (biến dự báo). Việc này giúp hệ thống học máy hiểu được mối quan hệ giữa các đặc trưng đầu vào và kết quả đầu ra mà chúng ta mong muốn dự đoán

```
[ ]  target = 'Class'
     predictors = ['Time', 'V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8', 'V9', 'V10',\
            'V11', 'V12', 'V13', 'V14', 'V15', 'V16', 'V17', 'V18', 'V19',\
            'V20', 'V21', 'V22', 'V23', 'V24', 'V25', 'V26', 'V27', 'V28',\
            'Amount']
```

Hình 5.1: Khai báo biến dự báo và biến mục tiêu

5.1.2. Chia dữ liệu thành tập huấn luyện, tập kiểm tra và tập xác thực:

Sau khi xác định các biến đầu vào và đầu ra, bước tiếp theo trong quá trình xây dựng mô hình là chia dữ liệu thành các tập riêng biệt để huấn luyện, kiểm tra và đánh giá mô hình. Việc chia dữ liệu giúp đảm bảo rằng mô hình học được từ một phần dữ liệu và được kiểm tra trên phần dữ liệu chưa từng thấy, từ đó phản ánh chính xác khả năng tổng quát hóa của mô hình.

```
[ ]  train_df, test_df = train_test_split(data_df, test_size=TEST_SIZE, random_state=RANDOM_STATE, shuffle=True )
     train_df, valid_df = train_test_split(train_df, test_size=VALID_SIZE, random_state=RANDOM_STATE, shuffle=True )
```

Hình 5.2: Chia tập dữ liệu

Trong đó:

- Tập huấn luyện (train set): được sử dụng để mô hình học các mối quan hệ giữa đầu vào và đầu ra.
- Tập kiểm tra (test set): được sử dụng để đánh giá hiệu suất cuối cùng của mô hình.

- Tập xác thực (validation set): được sử dụng trong quá trình huấn luyện để tinh chỉnh các siêu tham số và lựa chọn mô hình tốt nhất.

5.2. Mô Hình RandomForestClassifier.

5.2.1. Xác định tham số mô hình:

Mô hình Random Forest Classifier – một thuật toán học máy dựa trên tập hợp nhiều cây quyết định (decision trees) để cải thiện độ chính xác dự đoán và giảm hiện tượng quá khớp (overfitting).

```
clf = RandomForestClassifier(n_jobs=NO_JOBS,
                             random_state=RANDOM_STATE,
                             criterion=RFC_METRIC,
                             n_estimators=NUM_ESTIMATORS,
                             verbose=False)
```

```
[ ] clf.fit(train_df[predictors], train_df[target].values)
```

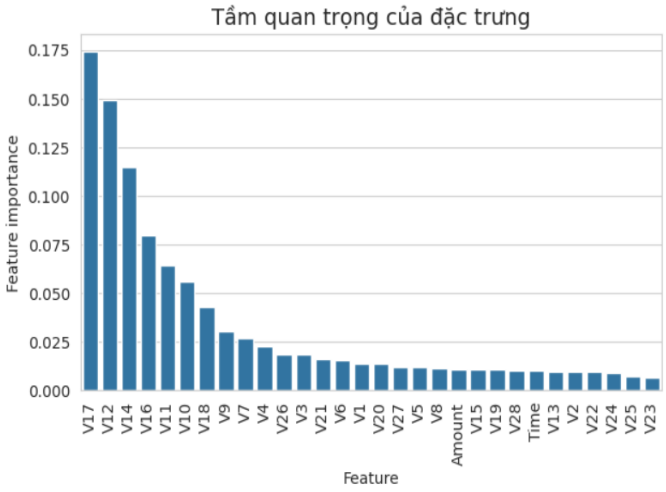
```
[ ] preds = clf.predict(valid_df[predictors])
```

Hình 5.3: Khởi tạo và huấn luyện mô hình

5.2.2. Tầm quan trọng của các đặc trưng

Sau khi huấn luyện mô hình, tiến hành đánh giá tầm quan trọng của các đặc trưng đầu vào (feature importance). Việc này giúp xác định xem yếu tố nào ảnh hưởng nhiều nhất đến quyết định của mô hình.

```
[ ] tmp = pd.DataFrame({'Feature': predictors, 'Feature importance': clf.feature_importances_})
    tmp = tmp.sort_values(by='Feature importance',ascending=False)
    plt.figure(figsize = (7,4))
    plt.title('Tầm quan trọng của đặc trưng',fontsize=14)
    s = sns.barplot(x='Feature',y='Feature importance',data=tmp)
    s.set_xticklabels(s.get_xticklabels(),rotation=90)
    plt.show()
```

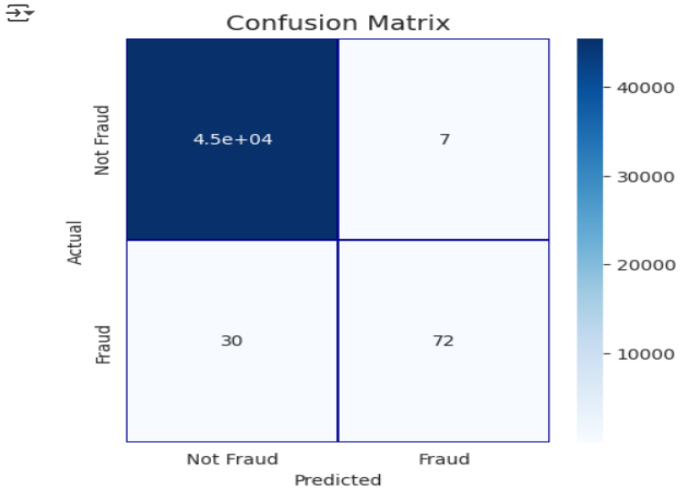


Hình 5.4: Biểu đồ thể hiện tầm quan trọng của đặc trưng

5.2.3. Ma trận nhầm lẫn:

Ma trận nhầm lẫn được sử dụng để đánh giá hiệu quả phân loại của mô hình, thể hiện số lượng dự đoán đúng và sai trên từng lớp (Not Fraud và Fraud):

```
[ ]  cm = pd.crosstab(valid_df[target].values, preds, rownames=['Actual'], colnames=['Predicted'])
     fig, (ax1) = plt.subplots(ncols=1, figsize=(5,5))
     sns.heatmap(cm,
                 xticklabels=['Not Fraud', 'Fraud'],
                 yticklabels=['Not Fraud', 'Fraud'],
                 annot=True,ax=ax1,
                 linewidths=.2,linecolor="Darkblue", cmap="Blues")
     plt.title('Confusion Matrix', fontsize=14)
     plt.show()
```



Hình 5.5: Ma trận nhầm lẫn của mô hình

Thông qua ma trận này, ta có thể đánh giá được độ chính xác, tỷ lệ phát hiện gian lận và tỷ lệ báo động giả.

5.3. Mô Hình AdaBoostClassifier.

5.3.1. Chuẩn bị mô hình và huyến luyện mô hình:

AdaBoost (Adaptive Boosting) là một thuật toán học máy dạng ensemble, kết hợp nhiều mô hình yếu (weak learners) – thường là các cây quyết định nông – để tạo ra một mô hình mạnh có độ chính xác cao hơn.

```
[ ]  clf = AdaBoostClassifier(random_state=RANDOM_STATE,
                              algorithm='SAMME',
                              learning_rate=0.8,
                                  n_estimators=NUM_ESTIMATORS)
```

```
[ ]  clf.fit(train_df[predictors], train_df[target].values)
```

```
[ ]  preds = clf.predict(valid_df[predictors])
```

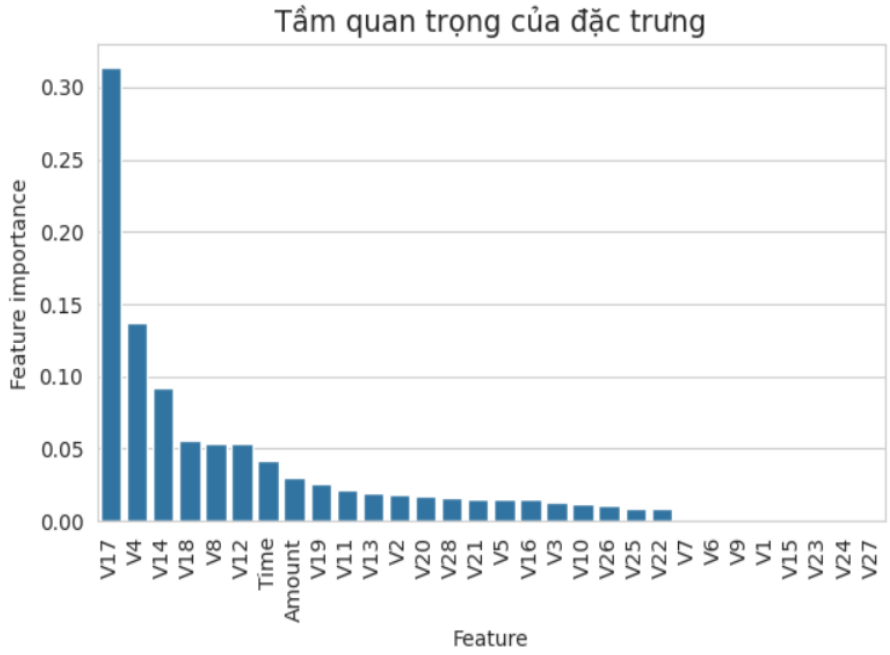
Hình 5.6: Khởi tạo và dự đoán mô hình

Mục tiêu là tăng khả năng phát hiện gian lận từ các mô hình học yếu kết hợp lại.

5.3.2: Tầm quan trọng của đặc trưng:

Biểu đồ giúp nhận biết được đặc trưng nào ảnh hưởng lớn nhất đến việc phân loại giao dịch.

```
[ ]  tmp = pd.DataFrame({'Feature': predictors, 'Feature importance': clf.feature_importances_})
     tmp = tmp.sort_values(by='Feature importance',ascending=False)
     plt.figure(figsize = (7,4))
     plt.title('Tầm quan trọng của đặc trưng',fontsize=14)
     s = sns.barplot(x='Feature',y='Feature importance',data=tmp)
     s.set_xticklabels(s.get_xticklabels(),rotation=90)
     plt.show()
```

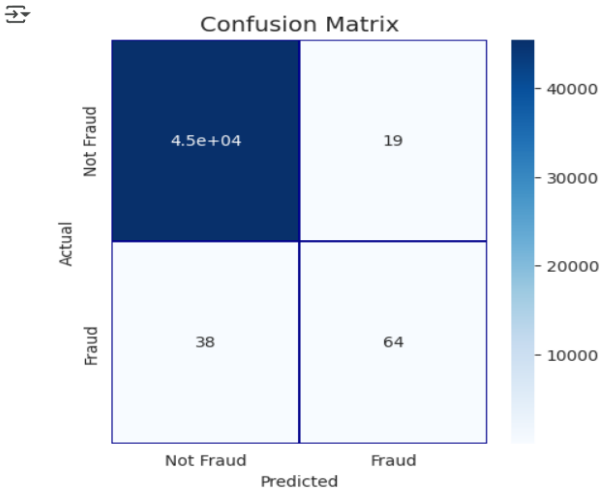


Hình 5.7: Biểu đồ tầm quan trọng của các đặc trưng trong mô hình AdaBoost

5.3.3. Ma trận nhầm lẫn:

Để đánh giá hiệu suất mô hình, ta sử dụng ma trận nhầm lẫn:

```
[ ] cm = pd.crosstab(valid_df[target].values, preds, rownames=['Actual'], colnames=['Predicted'])
    fig, (ax1) = plt.subplots(ncols=1, figsize=(5,5))
    sns.heatmap(cm,
                xticklabels=['Not Fraud', 'Fraud'],
                yticklabels=['Not Fraud', 'Fraud'],
                annot=True,ax=ax1,
                linewidths=.2,linecolor="Darkblue", cmap="Blues")
    plt.title('Confusion Matrix', fontsize=14)
    plt.show()
```



Hình 5.8: Ma trận nhầm lẫn của mô hình AdaBoost

Thông qua ma trận này, ta đánh giá được khả năng mô hình:

- Phát hiện đúng các giao dịch gian lận.
- Hạn chế cảnh báo sai đối với giao dịch hợp lệ.

5.4. Mô Hình CatBoostClassifier.

5.4.1. Chuẩn bị mô hình và huyến luyện mô hình:

CatBoost là một thuật toán học máy hiện đại thuộc họ Boosting, được phát triển bởi Yandex. Đây là mô hình tối ưu cho dữ liệu dạng bảng (tabular data) và đặc biệt hiệu quả ngay cả khi không cần xử lý đặc biệt cho dữ liệu đầu vào.

```
[ ] clf = CatBoostClassifier(iterations=500,
                             learning_rate=0.02,
                             depth=12,
                             eval_metric='AUC',
                             random_seed = RANDOM_STATE,
                             bagging_temperature = 0.2,
                             od_type='Iter',
                             metric_period = VERBOSE_EVAL,
                             od_wait=100)
```

```
[ ] clf.fit(train_df[predictors], train_df[target].values,verbose=True)
```

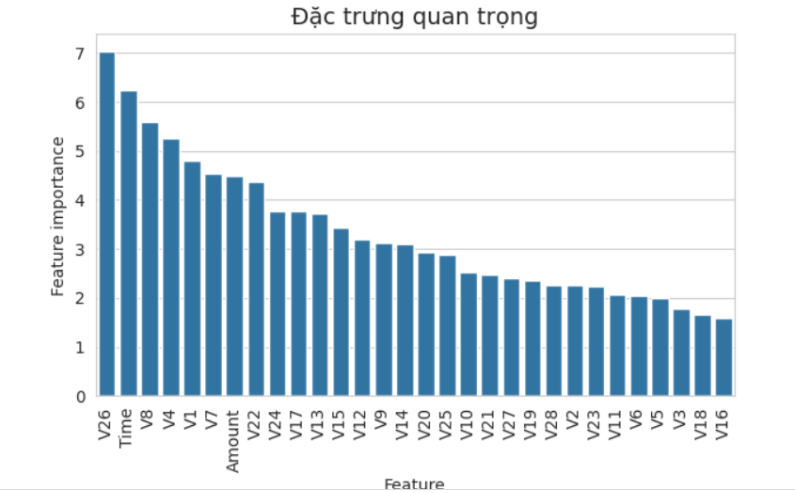
```
[ ] preds = clf.predict(valid_df[predictors])
```

Hình 5.9: Khởi tạo và dự đoán mô hình

5.4.2. Tầm quan trọng của các đặc trưng:

CatBoost hỗ trợ phân tích mức độ quan trọng của từng đặc trưng, giúp hiểu rõ mô hình đang dựa vào những yếu tố nào để đưa ra quyết định:

```
[ ] tmp = pd.DataFrame({'Feature': predictors, 'Feature importance': clf.feature_importances_})
    tmp = tmp.sort_values(by='Feature importance',ascending=False)
    plt.figure(figsize = (7,4))
    plt.title('Đặc trưng quan trọng',fontsize=14)
    s = sns.barplot(x='Feature',y='Feature importance',data=tmp)
    s.set_xticklabels(s.get_xticklabels(),rotation=90)
    plt.show()
```

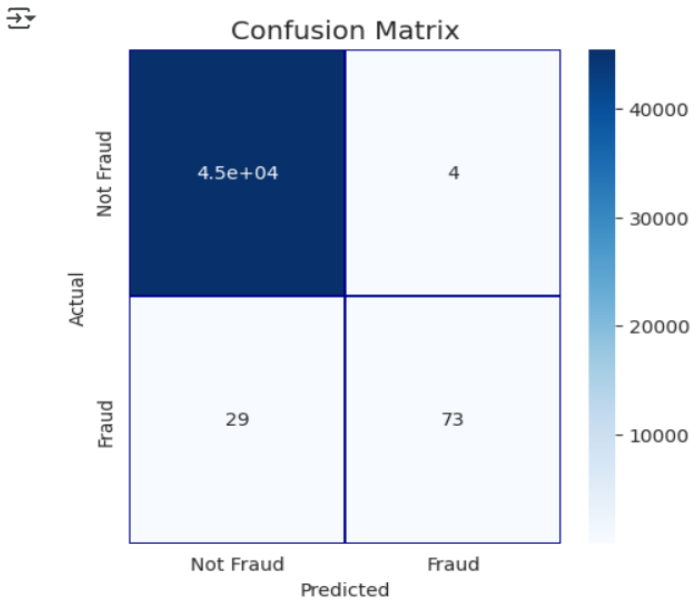


Hình 5.10: Biểu đồ tầm quan trọng của đặc trưng (CatBoost)

5.4.3. Ma trận nhầm lẫn:

Ma trận nhầm lẫn thể hiện chi tiết số lượng giao dịch được dự đoán đúng và sai theo từng lớp:

```
cm = pd.crosstab(valid_df[target].values, preds, rownames=['Actual'], colnames=['Predicted'])
fig, (ax1) = plt.subplots(ncols=1, figsize=(5,5))
sns.heatmap(cm,
            xticklabels=['Not Fraud', 'Fraud'],
            yticklabels=['Not Fraud', 'Fraud'],
            annot=True,ax=ax1,
            linewidths=.2,linecolor="Darkblue", cmap="Blues")
plt.title('Confusion Matrix', fontsize=14)
plt.show()
```



Hình 5.11: Ma trận nhầm lẫn của mô hình CatBoost

Nhờ vào khả năng học sâu, CatBoost thường cho tỷ lệ phát hiện gian lận (TP) cao hơn, đồng thời giảm số lượng cảnh báo sai (FP).

5.5. Mô Hình XGBoost.

5.5.1. Chuẩn bị mô hình và huyến luyện mô hình:

XGBoost (Extreme Gradient Boosting) là một trong những mô hình boosting hiệu quả và phổ biến nhất hiện nay, nổi bật nhờ tốc độ huấn luyện nhanh và khả năng tối ưu hóa cao trong các bài toán phân loại.

```
[ ]  # Chuẩn bị tập dữ liệu train và valid
     dtrain = xgb.DMatrix(train_df[predictors], train_df[target].values)
     dvalid = xgb.DMatrix(valid_df[predictors], valid_df[target].values)
     dtest = xgb.DMatrix(test_df[predictors], test_df[target].values)

     # Theo dõi chỉ số nào (trong trường hợp này là **train** và **valid**)
     watchlist = [(dtrain, 'train'), (dvalid, 'valid')]

     # Thiết lập tham số cho xgboost
     params = {}
     params['objective'] = 'binary:logistic'
     params['eta'] = 0.039
     params['silent'] = True
     params['max_depth'] = 2
     params['subsample'] = 0.8
     params['colsample_bytree'] = 0.9
     params['eval_metric'] = 'auc'
     params['random_state'] = RANDOM_STATE
```

```
model = xgb.train(params,
                  dtrain,
                  MAX_ROUNDS,
                  watchlist,
                  early_stopping_rounds=EARLY_STOP,
                  maximize=True,
                  verbose_eval=VERBOSE_EVAL)
```

Hình 5.12: Khởi tạo và dự đoán mô hình

- Mô hình đạt AUC rất cao từ những vòng đầu, và đạt trạng thái ổn định nhờ kỹ thuật **early stopping**.

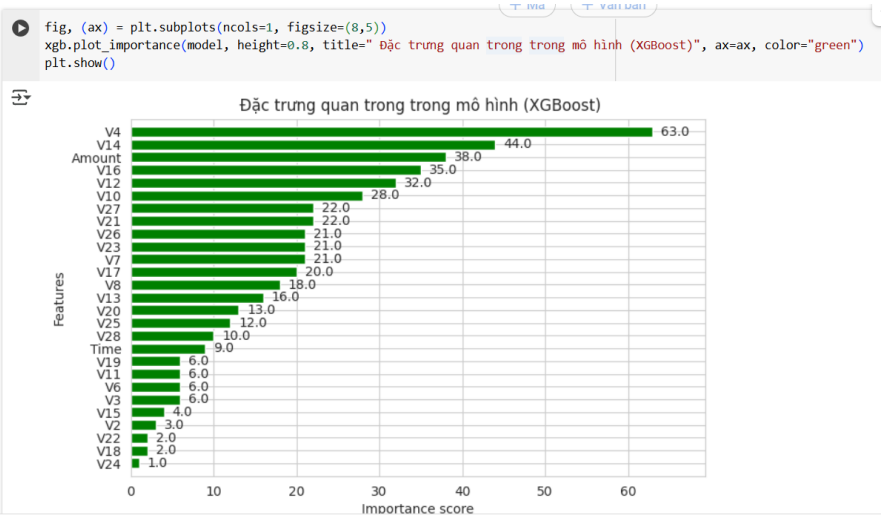
```
[ ]  preds = model.predict(dtest)
```

Hình 5.13: Dự đoán kết quả của mô hình

5.5.2. Tầm quan trọng của các đặc trưng:

Biểu đồ dưới đây cho thấy mức độ quan trọng của từng đặc trưng (feature importance) mà XGBoost sử dụng để phân loại giao dịch:

```
fig, (ax) = plt.subplots(ncols=1, figsize=(8,5))
xgb.plot_importance(model, height=0.8, title=" Đặc trưng quan trong trong mô hình (XGBoost)", ax=ax, color="green")
plt.show()
```



Hình 5.14: Biểu đồ đặc trưng quan trọng trong mô hình XGBoost

5.6: Mô Hình LightGBM.

5.6.1. Xác định tham số và chuẩn bị mô hình:

LightGBM (Light Gradient Boosting Machine) là một mô hình boosting tiên tiến, tối ưu về tốc độ và bộ nhớ, rất phù hợp với dữ liệu lớn và nhiều chiều.

```
[ ] params = {
        'boosting_type': 'gbdt',
        'objective': 'binary',
        'metric': 'auc',
        'learning_rate': 0.05,
        'num_leaves': 7,  # nên để nhỏ hơn 2^(max_depth)
        'max_depth': 4,  # -1 nghĩa là không giới hạn
        'min_child_samples': 100,  # Số lượng mẫu tối thiểu cần có trong một node lá (min_data_in_leaf)
        'max_bin': 100,  # Số lượng bin được chia cho giá trị của đặc trưng
        'subsample': 0.9,  # Tỉ lệ mẫu được lấy ngẫu nhiên từ tập huấn luyện
        'subsample_freq': 1,  # Tần suất lấy mẫu, <=0 nghĩa là không sử dụng
        'colsample_bytree': 0.7,  # Tỉ lệ cột được lấy khi xây mỗi cây
        'min_child_weight': 0,  # Tổng trọng số (hessian) tối thiểu cần có trong một node lá
        'min_split_gain': 0,  # Giá trị tối thiểu của lambda_l1, lambda_l2 hoặc min_gain_to_split để chia node
        'nthread': 8,
        'verbose': 0,
        'scale_pos_weight': 150,  # vì dữ liệu huấn luyện rất mất cân bằng
       }
```

```
[ ]  dtrain = lgb.Dataset(train_df[predictors].values,
                          label=train_df[target].values,
                          feature_name=predictors)

     dvalid = lgb.Dataset(valid_df[predictors].values,
                          label=valid_df[target].values,
                          feature_name=predictors)
```

```
[ ]  preds = model.predict(test_df[predictors])
```

Hình 5.15: Khởi tạo và dự đoán mô hình

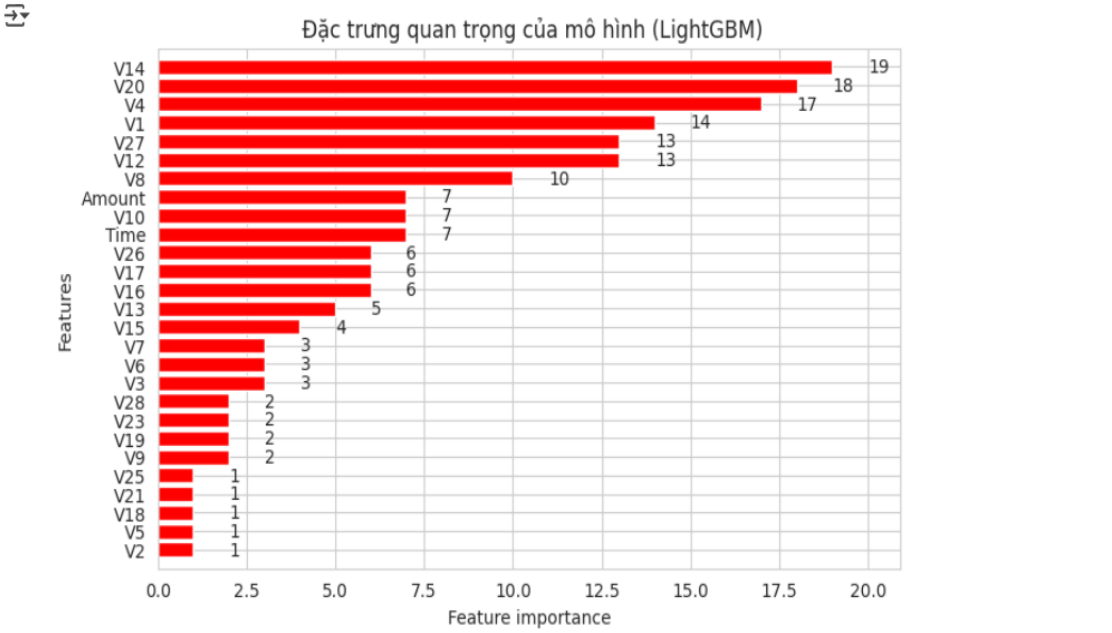
5.6.2. Đặc trưng quan trọng của mô hình:

- Biểu đồ dưới thể hiện mức độ ảnh hưởng của từng đặc trưng tới dự đoán của mô hình

```
evals_results = {}

model = lgb.train(params,
                  dtrain,
                  valid_sets=[dtrain, dvalid],
                  valid_names=['train','valid'],
                  num_boost_round=MAX_ROUNDS,
                  callbacks=[lgb.early_stopping(stopping_rounds=2*EARLY_STOP, verbose=VERBOSE_EVAL)],
                  feval=None)
```

```
fig, (ax) = plt.subplots(ncols=1, figsize=(8,5))
lgb.plot_importance(model, height=0.8, title=" Đặc trưng quan trọng của mô hình (LightGBM)", ax=ax,color="red")
plt.show()
```



Hình 5.16: Biểu đồ tầm quan trọng của đặc trưng trong LightGBM

- Các đặc trưng như V14, V10, Amount, V17 tiếp tục giữ vai trò quan trọng.
- Điều này giúp nhóm giải thích được vì sao mô hình đưa ra quyết định phân loại gian lận.

5.6.3. Huấn luyện và đánh giá mô hình bằng kỹ thuật kiểm định chéo:

Mô hình được đánh giá độ ổn định thông qua kỹ thuật K-Fold Cross Validation với 5 lần gập (5 folds)

```
[ ]  kf = KFold(n_splits = NUMBER_KFOLDS, random_state = RANDOM_STATE, shuffle = True)

     # Tạo mảng và DataFrame để lưu kết quả
     oof_preds = np.zeros(train_df.shape[0])
     test_preds = np.zeros(test_df.shape[0])
     feature_importance_df = pd.DataFrame()
     n_fold = 0
     for train_idx, valid_idx in kf.split(train_df):
         train_x, train_y = train_df[predictors].iloc[train_idx],train_df[target].iloc[train_idx]
         valid_x, valid_y = train_df[predictors].iloc[valid_idx],train_df[target].iloc[valid_idx]
```

```
        evals_results = {}
        model =  LGBMClassifier(
                     nthread=-1,
                     n_estimators=2000,
                     learning_rate=0.01,
                     num_leaves=80,
                     colsample_bytree=0.98,
                     subsample=0.78,
                     reg_alpha=0.04,
                     reg_lambda=0.073,
                     subsample_for_bin=50,
                     boosting_type='gbdt',
                     is_unbalance=False,
                     min_split_gain=0.025,
                     min_child_weight=40,
                     min_child_samples=510,
                     objective='binary',
                     metric='auc',
                     silent=-1,
                     verbose=-1,
                     feval=None)
        model.fit(train_x, train_y, eval_set=[(train_x, train_y), (valid_x, valid_y)],
                    eval_metric= 'auc', callbacks=[lgb.early_stopping(stopping_rounds=EARLY_STOP, verbose=VERBOSE_EVAL)])

        oof_preds[valid_idx] = model.predict_proba(valid_x, num_iteration=model.best_iteration_)[:, 1]
        test_preds += model.predict_proba(test_df[predictors], num_iteration=model.best_iteration_)[:, 1] / kf.n_splits
```

```
        fold_importance_df = pd.DataFrame()
        fold_importance_df["feature"] = predictors
        fold_importance_df["importance"] = clf.feature_importances_
        fold_importance_df["fold"] = n_fold + 1

        feature_importance_df = pd.concat([feature_importance_df, fold_importance_df], axis=0)
        print('Fold %2d AUC : %.6f' % (n_fold + 1, roc_auc_score(valid_y, oof_preds[valid_idx])))
        del model, train_x, train_y, valid_x, valid_y
        gc.collect()
        n_fold = n_fold + 1
    train_auc_score = roc_auc_score(train_df[target], oof_preds)
    print('Full AUC score %.6f' % train_auc_score)
```

Hình 5.17: Chỉ số AUC đánh giá khả năng phân biệt giữa giao dịch gian lận và không gian lận

```
Training until validation scores don't improve for 50 rounds
Early stopping, best iteration is:
[231]   training's auc: 0.975418        valid_1's auc: 0.998184
Fold  1 AUC : 0.998184
Training until validation scores don't improve for 50 rounds
Early stopping, best iteration is:
[44]    training's auc: 0.979318        valid_1's auc: 0.966302
Fold  2 AUC : 0.966302
Training until validation scores don't improve for 50 rounds
Early stopping, best iteration is:
[202]   training's auc: 0.981964        valid_1's auc: 0.943196
Fold  3 AUC : 0.943196
Training until validation scores don't improve for 50 rounds
Early stopping, best iteration is:
[69]    training's auc: 0.971829        valid_1's auc: 0.993275
Fold  4 AUC : 0.993275
Training until validation scores don't improve for 50 rounds
Early stopping, best iteration is:
[126]   training's auc: 0.972246        valid_1's auc: 0.984831
Fold  5 AUC : 0.984831
Full AUC score 0.970312
```

Hình 5.18: Kết quả đánh giá của chỉ số AUC

- AUC trung bình toàn bộ: 0.9703 (~97.03%)
- Điều này chứng tỏ mô hình LightGBM hoạt động ổn định, độ lệch giữa các fold thấp, giúp tăng độ tin cậy khi triển khai vào thực tế.

CHƯƠNG 6: SO SÁNH KẾT QUẢ VÀ ĐÁNH GIÁ

6.1. Kết quả huyến luyện mô hình.

Sau khi tiến hành huấn luyện và đánh giá năm mô hình khác nhau: RandomForest, AdaBoost, CatBoost, XGBoost và LightGBM trên cùng một bộ dữ liệu, nhóm đã ghi nhận lại các chỉ số đánh giá quan trọng như: AUC (Area Under Curve), hiệu suất trên tập kiểm tra, và ma trận nhầm lẫn, ta đã ghi nhận các kết quả đánh giá hiệu suất qua chỉ số AUC (Area Under Curve) như sau:

6.1.1. Mô hình Random Forest:

- Mô hình này cho kết quả AUC là 0.8529 trên tập kiểm tra. Đây là một thuật toán dễ sử dụng, hoạt động ổn định và không yêu cầu nhiều tinh chỉnh siêu tham số. Tuy nhiên, hiệu suất vẫn chưa đủ cao so với các mô hình boosting khác.

```
[ ]  roc_auc_score(valid_df[target].values, preds)

     np.float64(0.8528641975628091)
```

Hình 6.1: Kết quả (AUC - Area Under Curve) của Random Forest

6.1.2. Mô hình AdaBoost:

- AdaBoost là mô hình cho kết quả kém nhất trong số các mô hình được thử nghiệm, với AUC chỉ đạt 0.8135. Mô hình này có khả năng học kém hơn trong bài toán phân loại không cân bằng như phát hiện gian lận. Do đó, AdaBoost không được lựa chọn làm mô hình cuối cùng.

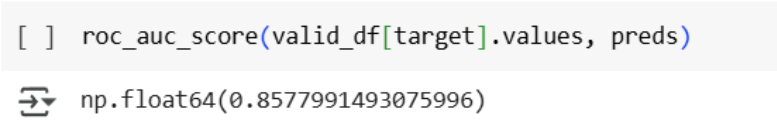
```
[ ]  roc_auc_score(valid_df[target].values, preds)

     np.float64(0.8135165474463918)
```

Hình 6.2: Kết quả (AUC - Area Under Curve) của AdaBoost

6.1.3. Mô Hình CatBoostClassifier:

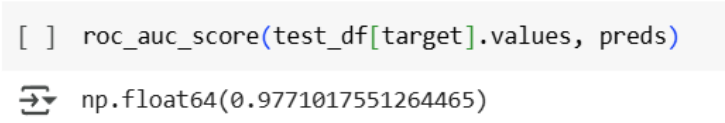
CatBoost đạt AUC khoảng 0.8578, cao hơn so với Random Forest và AdaBoost. Đây là mô hình boosting hiện đại, hỗ trợ tốt cho dữ liệu có đặc trưng phân loại. Tuy nhiên, kết quả vẫn chưa thực sự nổi bật để được chọn làm mô hình tốt nhất.



Hình 6.3: Kết quả (AUC - Area Under Curve) của CatBoost

6.1.4. Mô hình XGBoost:

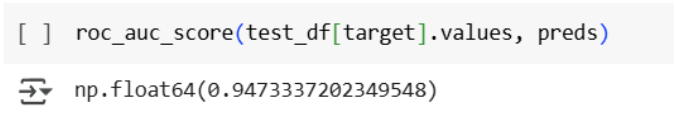
Đây là mô hình đạt kết quả cao nhất trong tất cả các mô hình đã thử nghiệm. Cụ thể, AUC đạt 0.9772 trên tập validation và 0.9771 trên tập kiểm tra (test set). Mô hình học nhanh, cho kết quả chính xác và không bị overfitting. Đây là mô hình được đánh giá tối ưu nhất.



Hình 6.4: Kết quả (AUC - Area Under Curve) của XGBoost

6.1.5. Mô hình LightGBM:

LightGBM đạt AUC 0.9579 trên tập validation và 0.9473 trên tập test. Ngoài ra, mô hình còn được đánh giá bằng kỹ thuật kiểm định chéo (cross-validation) với 5 lần gập, kết quả trung bình đạt 0.9703. Mô hình cho thấy hiệu suất tốt và độ ổn định cao, là lựa chọn phù hợp trong các hệ thống yêu cầu tốc độ hoặc giới hạn tài nguyên.



Hình 6.5: Kết quả (AUC - Area Under Curve) của LightGBM

6.2. Đánh giá mô hình tối ưu.

Dựa trên các kết quả thu được từ quá trình huấn luyện và đánh giá hiệu suất của 5 mô hình: Random Forest, AdaBoost, CatBoost, XGBoost và LightGBM, có thể đi đến kết luận về mô hình tối ưu như sau:

- XGBoost là mô hình cho kết quả dự đoán chính xác nhất trong tất cả các mô hình được thử nghiệm. Với chỉ số AUC đạt 0.9771 trên tập kiểm tra, XGBoost cho thấy khả năng phân loại cực kỳ hiệu quả giữa các giao dịch hợp lệ và gian lận. Bên cạnh đó, mô hình cũng thể hiện sự ổn định và khả năng tổng quát tốt, không xảy ra hiện tượng quá khớp (overfitting), đồng thời tương thích với các bài toán dữ liệu mất cân bằng.
- LightGBM là mô hình có hiệu suất rất gần với XGBoost, đặc biệt nổi bật về tốc độ huấn luyện và hiệu quả tính toán. Với AUC validation là 0.9579 và AUC test là 0.9473, kết hợp với kiểm định chéo đạt trung bình 0.9703, LightGBM là lựa chọn phù hợp trong các hệ thống cần cập nhật mô hình nhanh hoặc triển khai thực tế với tài nguyên hạn chế.
- Các mô hình còn lại như CatBoost, Random Forest, và AdaBoost tuy có thể ứng dụng trong các tình huống đơn giản, nhưng hiệu suất không đủ cao để đáp ứng yêu cầu của một hệ thống phát hiện gian lận chính xác và hiệu quả.

- Kết luận:

  XGBoost được lựa chọn là mô hình tối ưu nhất cho bài toán phát hiện gian lận trong giao dịch thẻ tín dụng, nhờ vào độ chính xác cao, hiệu suất ổn định và khả năng thích ứng tốt với dữ liệu mất cân bằng. Trong trường hợp cần thay thế mô hình với yêu cầu về tốc độ xử lý nhanh, LightGBM là lựa chọn dự phòng hợp lý.

CHƯƠNG 7: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1. Kết luận.

Trong đồ án này, ta đã nghiên cứu và xây dựng hệ thống phát hiện gian lận trong giao dịch thẻ tín dụng bằng cách áp dụng các mô hình học máy. Dữ liệu sử dụng là một tập dữ liệu thực tế, có tính mất cân bằng cao giữa số lượng giao dịch hợp lệ và gian lận. Để giải quyết bài toán, nhóm đã thực hiện các bước chính như sau:

- Tiền xử lý dữ liệu và chuẩn hóa đặc trưng.
- Lựa chọn các thuật toán học máy phù hợp để huấn luyện mô hình, bao gồm: Random Forest, AdaBoost, CatBoost, XGBoost và LightGBM.
- Đánh giá mô hình bằng nhiều chỉ số, trong đó trọng tâm là AUC (Area Under Curve) – một thước đo phù hợp cho bài toán phân loại nhị phân không cân bằng.
- Sử dụng thêm kỹ thuật kiểm định chéo (cross-validation) để đánh giá tính ổn định và khả năng tổng quát hóa của mô hình.

Kết quả thực nghiệm cho thấy:

- XGBoost là mô hình tối ưu nhất, đạt AUC rất cao trên cả tập validation và test (lên đến 0.9771), chứng tỏ khả năng phân biệt tốt giữa giao dịch gian lận và giao dịch hợp lệ.
- LightGBM tuy có hiệu suất thấp hơn một chút nhưng lại nổi bật về tốc độ và hiệu quả, phù hợp với môi trường triển khai thực tế cần xử lý dữ liệu nhanh và liên tục.
- Các mô hình còn lại (CatBoost, Random Forest, AdaBoost) có thể được ứng dụng tùy theo yêu cầu cụ thể, nhưng nhìn chung cho hiệu quả thấp hơn.

Việc áp dụng các mô hình học máy hiện đại đã giúp ta giải quyết tốt bài toán phát hiện gian lận, đồng thời đặt nền móng cho việc mở rộng, nâng cấp hệ thống trong tương lai.

7.2. Hướng phát triển.

Trong tương lai, để cải thiện độ chính xác và tính ứng dụng thực tiễn của hệ thống, đề xuất một số hướng phát triển như sau:

- Tăng cường kỹ thuật xử lý dữ liệu mất cân bằng: Áp dụng các kỹ thuật như SMOTE, ADASYN hoặc sử dụng các hàm mất mát có trọng số nhằm xử lý tốt hơn tỷ lệ gian lận thấp.
- Tích hợp dữ liệu thời gian thực (real-time): Triển khai mô hình theo thời gian thực để giám sát và phát hiện gian lận ngay khi giao dịch diễn ra.
- Kết hợp thêm các kỹ thuật học sâu (Deep Learning): Sử dụng mạng nơ-ron tích chập (CNN) hoặc mạng tuần tự (LSTM) để khai thác sâu hơn các mối quan hệ giữa các đặc trưng giao dịch.
- Tối ưu hóa mô hình qua AutoML: Tự động hóa quá trình lựa chọn siêu tham số và mô hình phù hợp nhằm tiết kiệm thời gian và nâng cao hiệu quả.
- Triển khai mô hình trên môi trường thật: Xây dựng hệ thống giám sát và phản hồi gian lận trực tiếp trên ứng dụng web hoặc hệ thống thanh toán thực tế.

TÀI LIỆU THAM KHẢO

[1] Kaggle, Credit Card Fraud Detection Dataset, 2013. [Trực tuyến]. Có tại: https://www.kaggle.com/mlg-ulb/creditcardfraud. [Truy cập: 10-08-2025].

[2] L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[3] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," Journal of Computer and System Sciences, vol. 55, no. 1, pp. 119–139, 1997.

[4] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, USA, 2016, pp. 785–794.

[5] G. Ke, Q. Meng, T. Finley, et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[6] L. Prokhorenkova, G. Gusev, A. Vorobev, et al., "CatBoost: unbiased boosting with categorical features," in Proceedings of the 32nd International Conference on Neural Information Processing Systems (NeurIPS), Montréal, Canada, 2018, pp. 6639–6649.

[7] T. Fawcett, "An introduction to ROC analysis," Pattern Recognition Letters, vol. 27, no. 8, pp. 861–874, 2006.

[8] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. MIT Press, 2016.

[9] Scikit-learn Developers, Scikit-learn: Machine Learning in Python, [Trực tuyến]. Có tại: https://scikit-learn.org/stable/. [Truy cập: 12-08-2025].

[10] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.